



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

13-存内计算加速器

贾弘洋 & 刘勇攀

2025/05/20



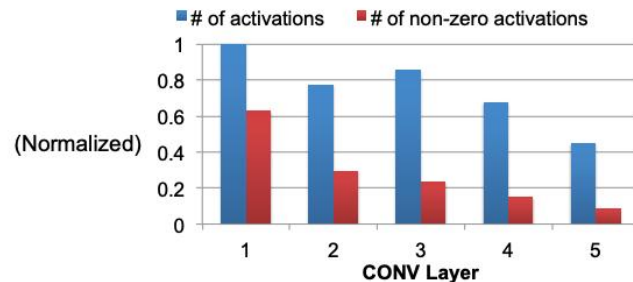
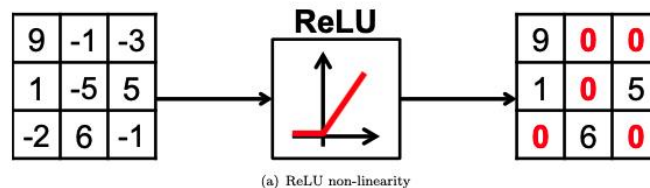
□ Activation Sparsity

□ ReLU function

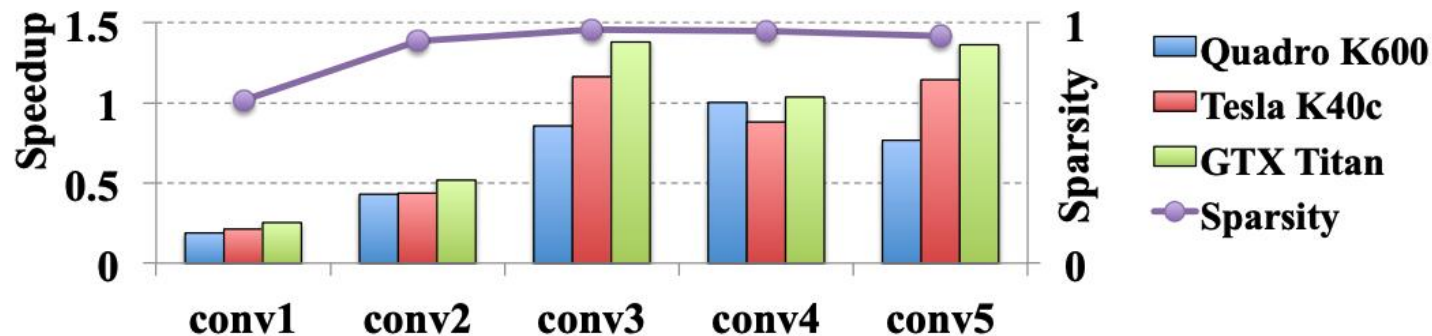
□ Sample from another network

□ Weight Sparsity

□ Network Prune

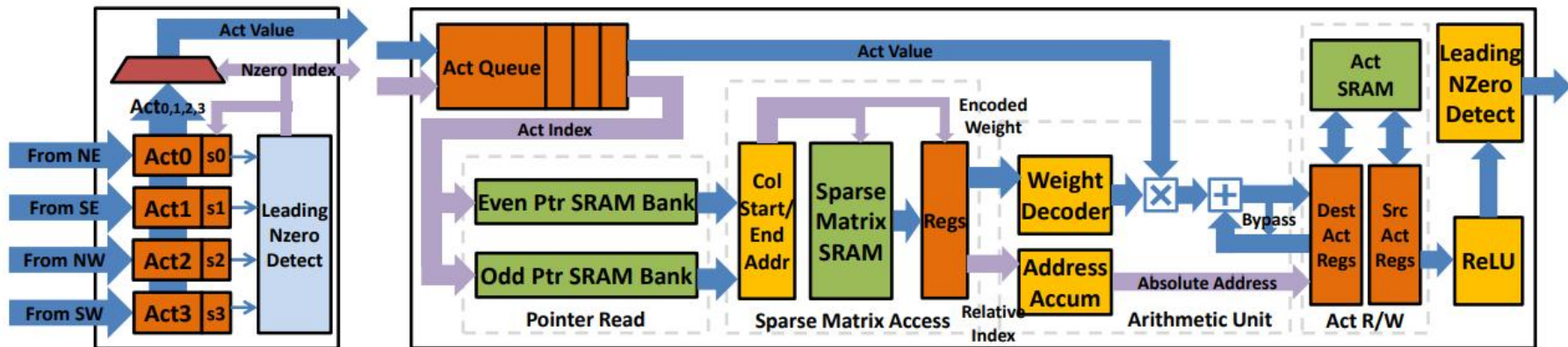


- Sparsity does not directly translate to speedup and data compression



- **Domain-specific accelerator** is needed

Previous work to utilize unstructured sparsity



- Sparse Storage Format
 - Compressed Sparse Column (CSC)
- Parallel PE Processing
 - Skip both clock cycles and energy

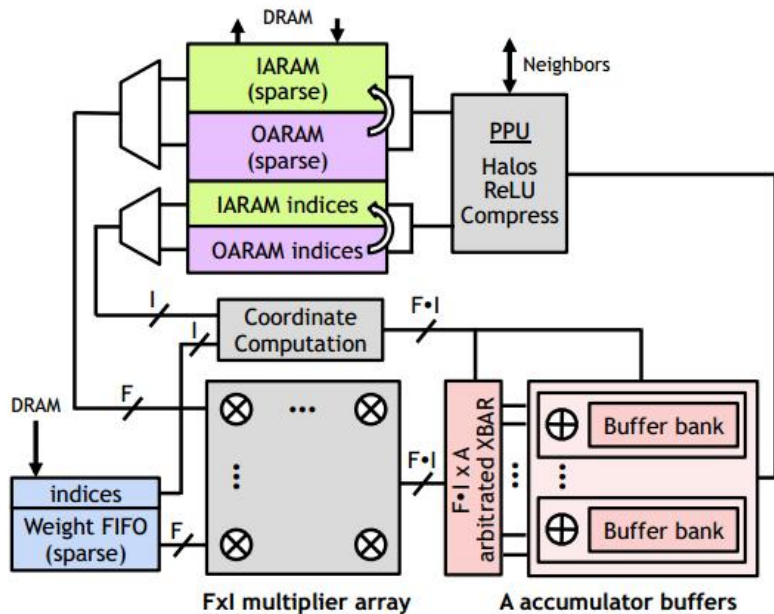
3400 $\times$ Energy Efficient than GeForce TITAN X

19 $\times$ Energy Efficient than DaDianNao

$$\begin{matrix} \vec{a} & (0 & 0 & a_2 & 0 & a_4 & a_5 & 0 & a_7) \end{matrix} \times \begin{matrix} \vec{b} & (b_0 & b_1 & -b_2 & b_3 & -b_4 & b_5 & -b_6 & -b_7 & b_8 & -b_9 & b_{10} & -b_{11} & -b_{12} & b_{13} & b_{14} & -b_{15}) \end{matrix}$$

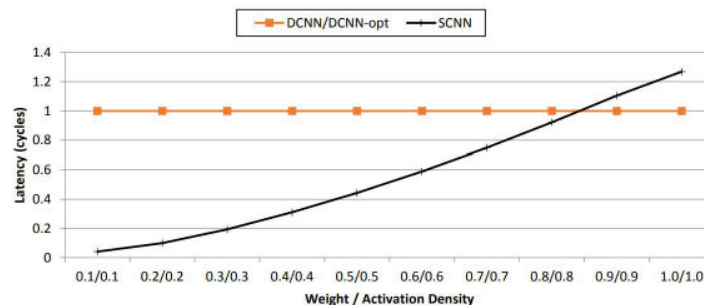
$$\begin{matrix} PE0 & w_{0,0} & 0 & w_{0,2} & 0 & w_{0,4} & w_{0,5} & w_{0,6} & 0 \\ PE1 & 0 & w_{1,1} & 0 & w_{1,3} & 0 & 0 & w_{1,6} & 0 \\ PE2 & 0 & 0 & w_{2,2} & 0 & w_{2,4} & 0 & 0 & w_{2,7} \\ PE3 & 0 & w_{3,1} & 0 & 0 & 0 & w_{0,5} & 0 & 0 \\ & w_{4,1} & 0 & 0 & w_{4,4} & 0 & 0 & 0 & 0 \\ & 0 & 0 & 0 & w_{5,4} & 0 & 0 & 0 & w_{5,7} \\ & 0 & 0 & 0 & 0 & w_{6,4} & 0 & w_{6,6} & 0 \\ w_{7,0} & 0 & 0 & 0 & w_{7,4} & 0 & 0 & w_{7,7} & 0 \\ w_{8,0} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & w_{8,7} \\ w_{9,0} & 0 & 0 & 0 & 0 & 0 & 0 & w_{9,6} & w_{9,7} \\ & 0 & 0 & 0 & 0 & w_{10,3} & 0 & 0 & 0 \\ & 0 & 0 & w_{11,2} & 0 & 0 & 0 & 0 & w_{11,7} \\ w_{12,0} & 0 & w_{12,2} & 0 & 0 & w_{12,5} & 0 & w_{12,7} & 0 \\ w_{13,0} & w_{13,2} & 0 & 0 & 0 & 0 & 0 & w_{13,6} & 0 \\ & 0 & 0 & w_{14,2} & w_{14,3} & w_{14,4} & w_{14,5} & 0 & 0 \\ & 0 & 0 & w_{15,2} & w_{15,3} & 0 & w_{15,5} & 0 & 0 \end{matrix} = \begin{matrix} \vec{b} & (b_0 & b_1 & -b_2 & b_3 & -b_4 & b_5 & -b_6 & -b_7 & b_8 & -b_9 & b_{10} & -b_{11} & -b_{12} & b_{13} & b_{14} & -b_{15}) \end{matrix} \xRightarrow{ReLU} \begin{matrix} \vec{b} & (b_0 & b_1 & 0 & b_3 & 0 & b_5 & 0 & 0 & 0 & 0 & b_{10} & 0 & 0 & b_{13} & b_{14} & 0) \end{matrix}$$

Review Previous work to utilize unstructured sparsity



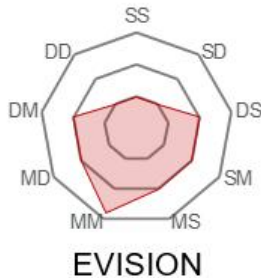
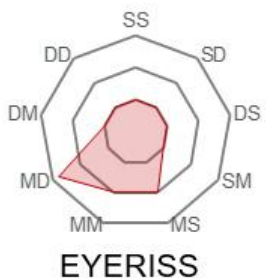
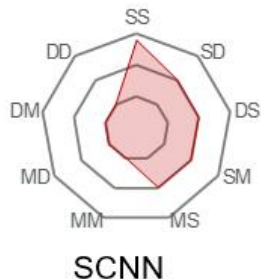
Parashar A, SCNN (ISCA2017)

- Index-Based Data Storage
 - Activation + Weight Sparse
- Sparse CNN Processing
 - Data Multiply + Index calculate
 - Hash-Based Accumulation Buffer



Review

Previous Solution: Incomplete Sparsity Coverage



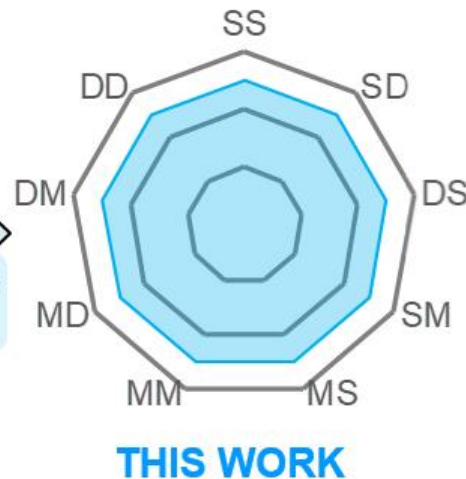
9 Typical sparsity mode

Weight	SD	MD	DD
	SM	MM	DM
	SS	MS	DS

Activation

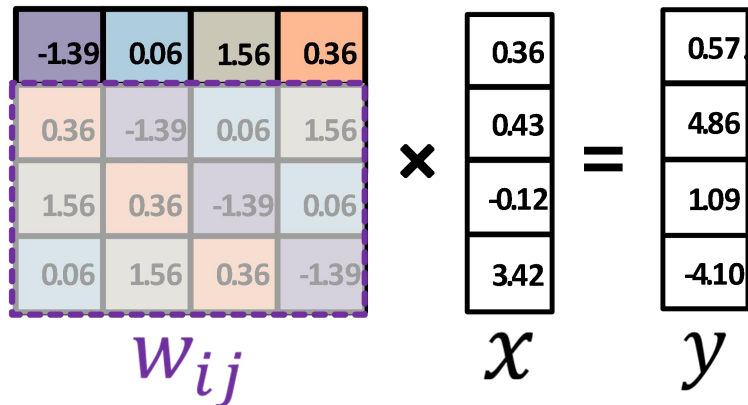
Design Target

9 sparsity mode
compatible

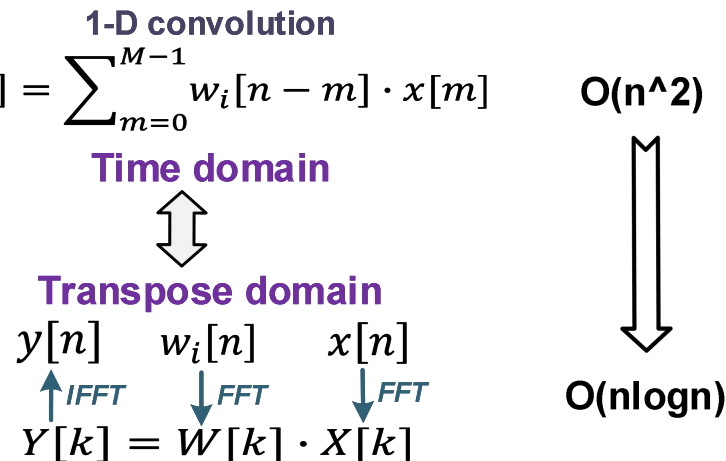


S: Sparse D: Dense M: Medium

Unified CNN/FC/RNN flow

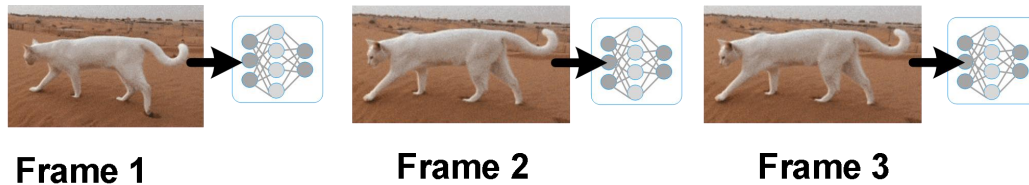


Transform-domain acceleration

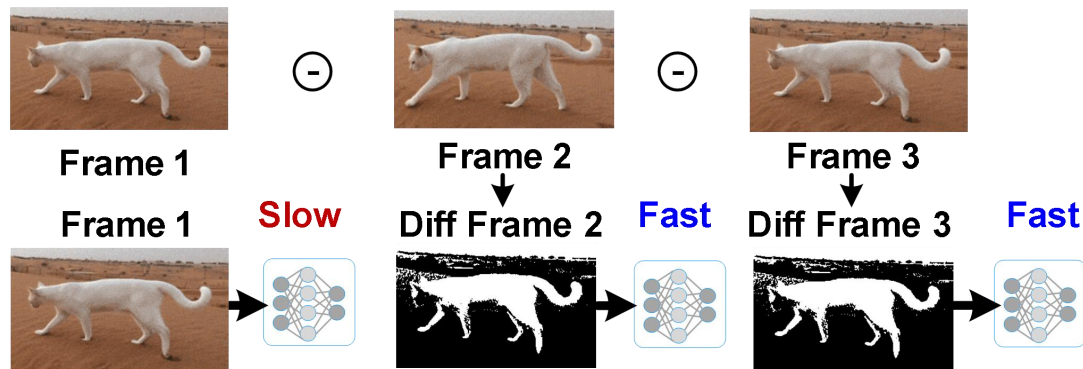


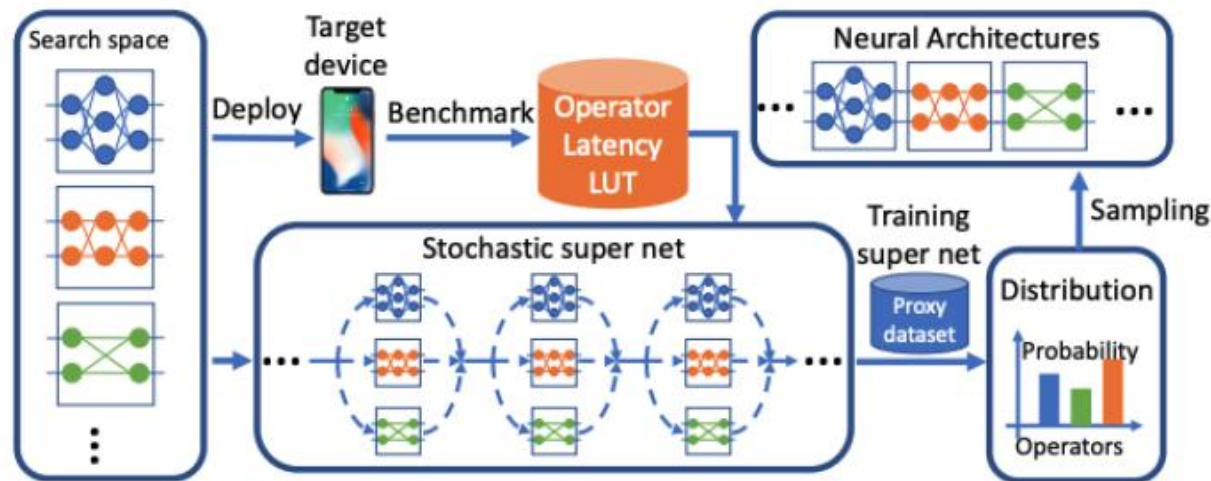
n (block-size)	8	16	32	64	128
$O(n^2)/O(n \log n)$	2.7x	4.0x	6.4x	10.7x	18.3x

Traditional Way
Process Full Frame



New Dimension
Process Differential Frame





Model	#Parameters	#FLOPs	Latency on iPhone X	Latency on Samsung S8	Top-1 acc (%)
FBNet-iPhoneX	4.47M	322M	19.84 ms (target)	23.33 ms	73.20
FBNet-S8	4.43M	293M	27.53 ms	22.12 ms (target)	73.27

Table 5. FBNets searched for different devices.

- ☐ **Sparsity overview**
- ☐ **By designing a domain-specific accelerator, we can achieve better performance.**
- ☐ **SW/HW codesign philosophy**
- ☐ **NAS is another choice to design efficient network**



- **Introduction**
- **Memory devices and computational primitives**
 - **Charge-based memory devices**
 - **Resistance-based memory devices**
- **SRAM-based CIM: from device to architecture**
 - **3 SRAM CIM primitives**
 - **Toward efficient MVM mapping to CIM array**
 - **Sparsity in CIM**
- **Challenges, Trade-offs and Metrics of CIM**

/Digital Systems: Trends and Opportunity



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

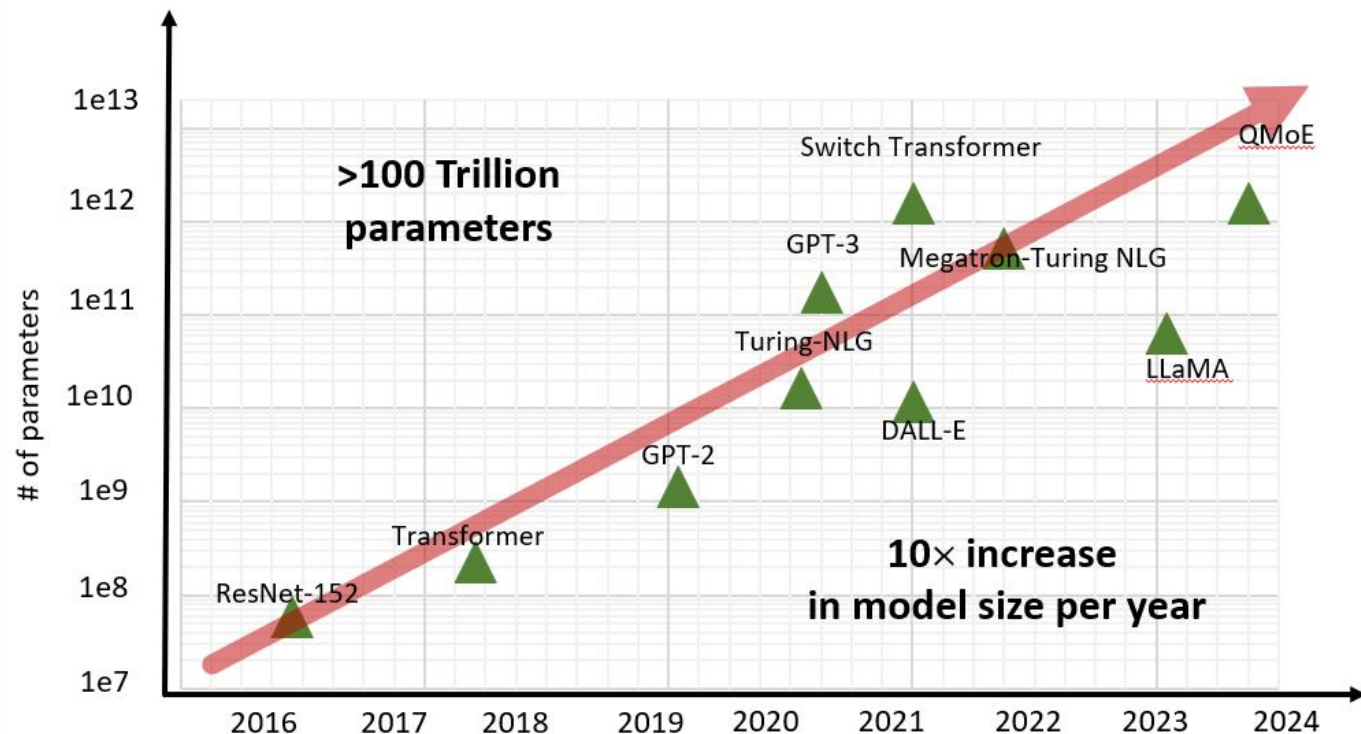
■ Three key trends

- ✓ Data access is a major bottleneck
- ✓ Energy consumption is a key limiter
- ✓ Energy to move data dominates compute energy

■ Opportunity

- ✓ Minimize data movement by performing computation directly (near) where the data resides
- ✓ Processing in memory (PIM)
 - In-memory computing
 - Near-memory computing/near data processing

/AI Era: compute demand increases rapidly

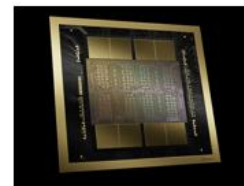


<https://epochai.org/data/epochdb/visualization>



AMD Zen4

CPU



Nvidia B200

GPU



Google TPU v5e

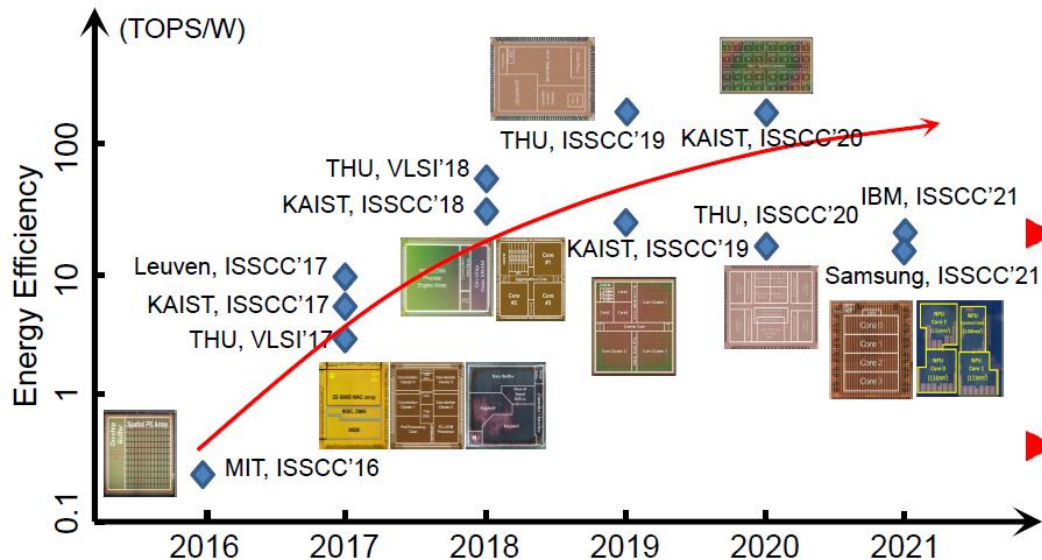
ASIC

Digital ASIC NN Processors



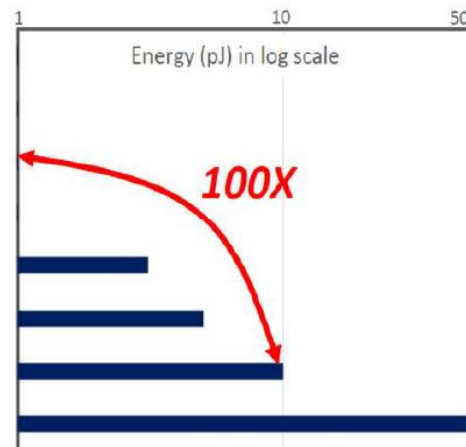
Traditional optimization strategies

- ❑ Data reuse
- ❑ Lower bit quantization
- ❑ Model compression (sparsity)



“Memory wall”

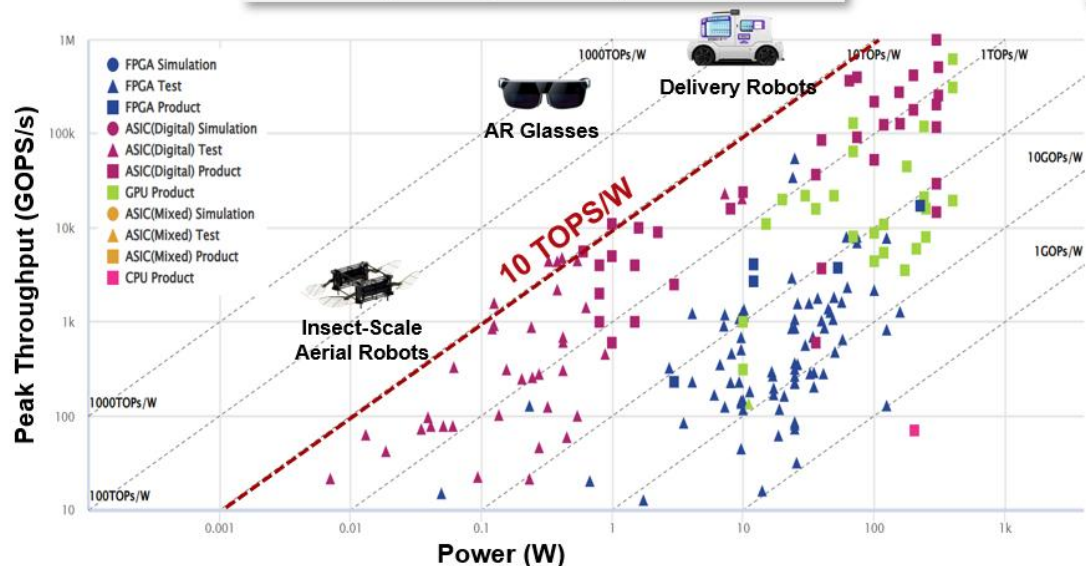
Operation	Energy (pJ)
Integer ADD (8b)	0.03
Integer ADD (16b)	0.05
Integer ADD (32b)	0.1
Integer MULT (8b)	0.2
Integer MULT (32b)	3.1
8KB SRAM Read (32b)	5
32KB SRAM Read (32b)	10
1MB SRAM Read (32b)	50



“Computing’s Energy Problem (and what we can do about it)”, M. Horowitz, ISSCC 2014

AI platforms face challenges

Efficiency of Digital Compute Platforms



<https://nicsefc.ee.tsinghua.edu.cn/projects/neural-network-accelerator/>

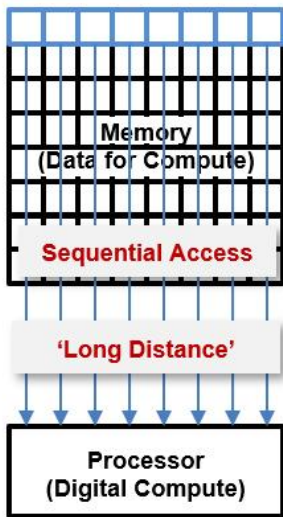
E.g., AI Models on the Edge

Model Type	GFLOPs per token
BERT base	0.09
BERT large	0.31
<u>LLaMA 7B</u>	7.0
<u>LLaMA 13B</u>	13.4
<u>LLaMA 35B</u>	33.5
<u>LLaMA 65B</u>	67.1

- Barriers seen for improving energy efficiency of conventional digital platforms: challenge of data movement

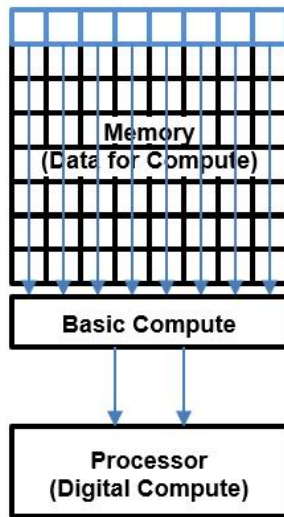
/ Moving computation closer to memory

Conventional Digital Architecture



- ❑ **'Long Distance'**
physical, logical, relative
- ❑ **Bandwidth mismatch**

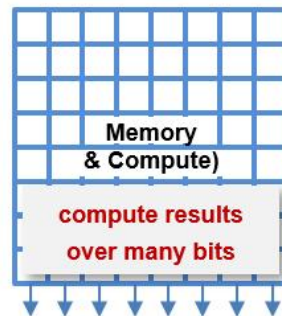
Near-memory/storage Computing



[UPMEM, *Hot Chips*'19]

- ❑ **Helps long distance mov.**
- ❑ **Same local mem. access**

In-memory Computing



[M. Kang, *ICASSP*'14]

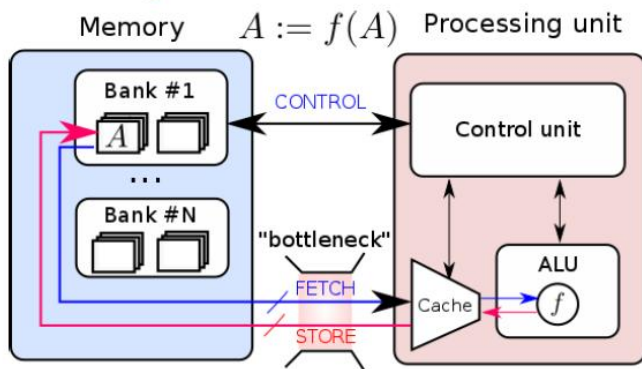
[J. Yue, *JSSC*'22]

[H. Jia, *JSSC*'22]

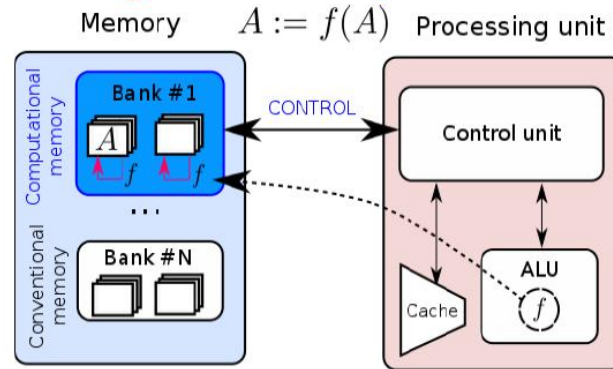
- ❑ **Enable 2D memory access**
- ❑ **Both row and col parallelism**

/ Compute in Memory (CIM)

Processing unit & Conventional memory



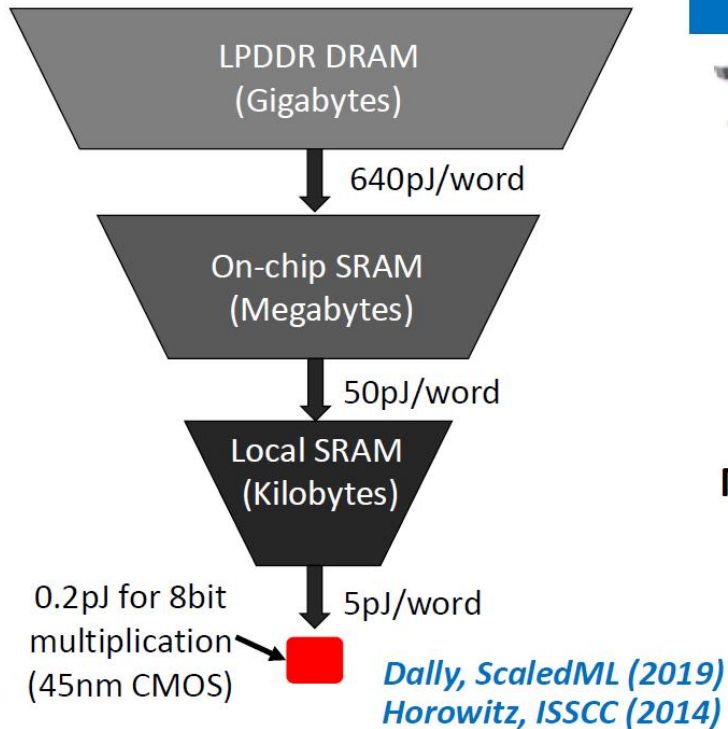
Processing unit & Computational memory



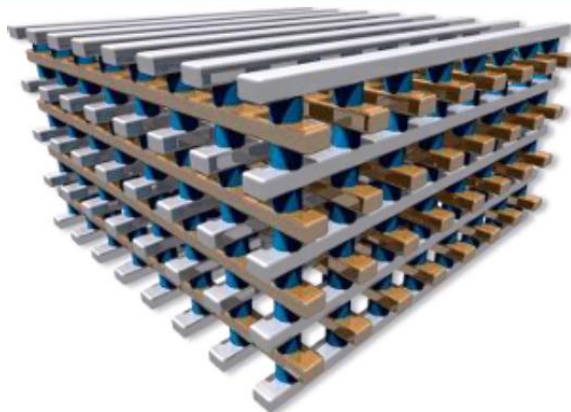
- Perform “certain” computational tasks **in place in memory**
- Achieved by exploiting **the physical attributes of the memory devices**, their **array level organization**, the **peripheral circuitry** as well as the **control logic**
- At **no point during computation**, the memory content is read back and **processed at the granularity of a single memory element**

/ Why CIM?

Reduce the cost of data motion



Reduce computational time complexity



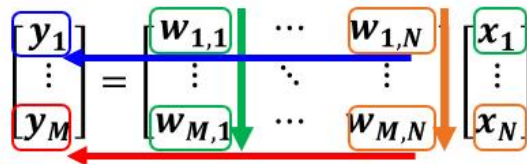
**Mostly from massive parallelism and
analog way of computing**
Additional complexity reduction from
physical coupling

*Sebastian et al., Nature Comm. (2017)
Di Ventra, Nature Phys. (2013)*

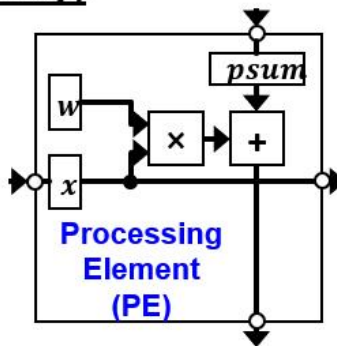
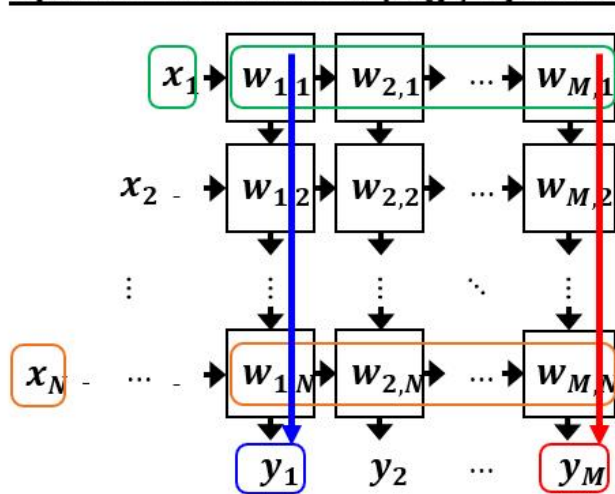
/ Spatial architectures to exploit 2-D data reuse

MVM:

$$\vec{y} = W \cdot \vec{x} \Rightarrow$$



Spatial Architecture (e.g., systolic array):



- Small, efficient distributed memory
- Minimize intermediate data movement

Reducing communication costs depends on computation locality vs. hardware density

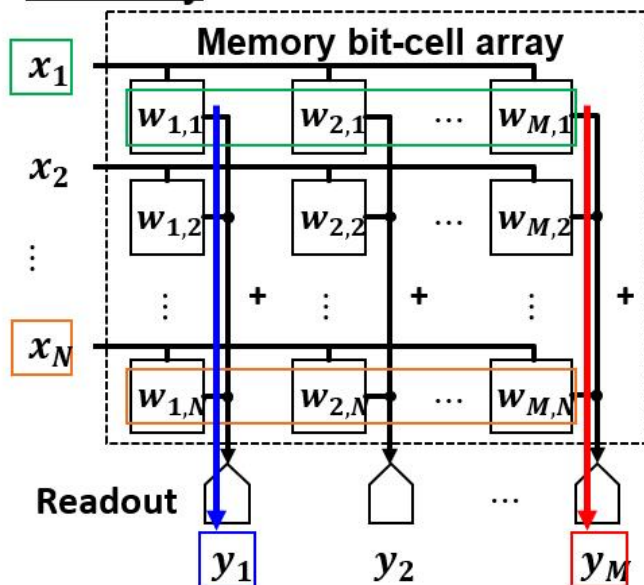
/ Functionality of IMC

MVM:

$$\vec{y} = W \cdot \vec{x} \Rightarrow$$

$$\begin{bmatrix} y_1 \\ \vdots \\ y_M \end{bmatrix} = \begin{bmatrix} w_{1,1} & \cdots & w_{1,N} \\ \vdots & \ddots & \vdots \\ w_{M,1} & \cdots & w_{M,N} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix}$$

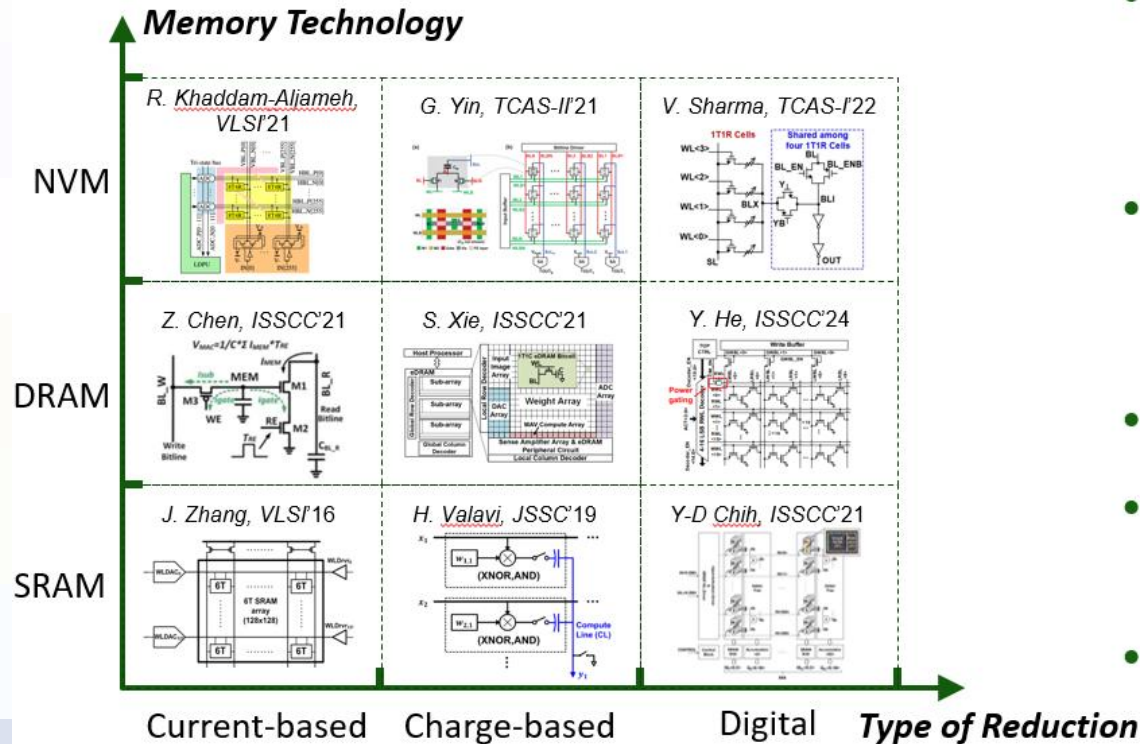
IMC Array



- **Bitcell:** both storage and compute
- Activate many/all rows at once, to access compute result over many bits, not single bits
- **Reduce memory access by N (parallelism along the bitline)**

/ A (rough) taxonomy of IMC

- Reduction operation
 - Computation related attributes
 - Current, charge and digital
- Memory technology
 - Storage related attributes
 - SRAM, DRAM, NVM...
- Many other ways to categorize
- Multiplication operation
 - Serial, parallel, look-up table...
- Number representation
 - Integer, floating point...



/ Typical Components in Analog CIM

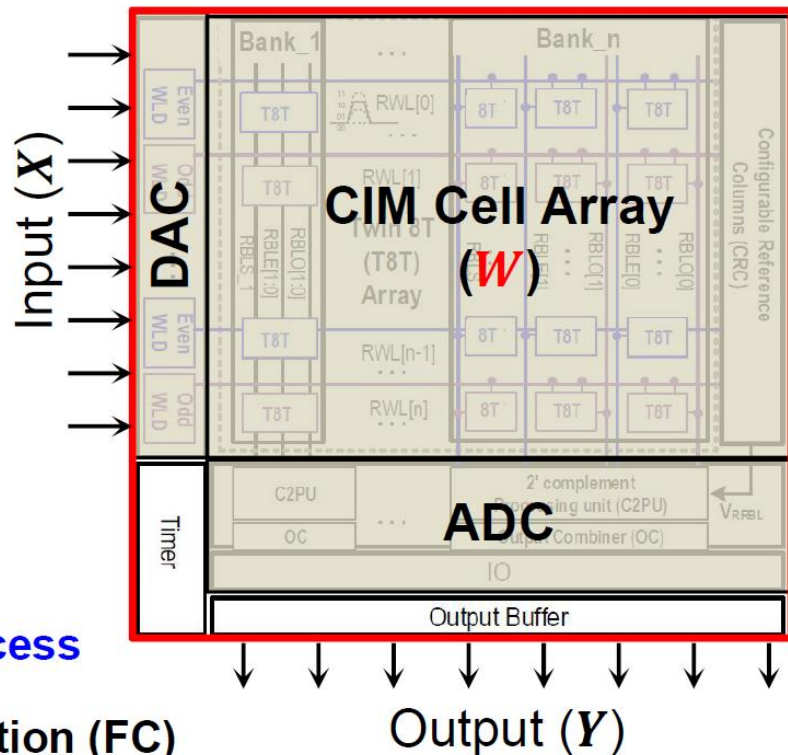
Main Modules: DAC, ADC, CIM Array

$$Y = W \cdot X$$

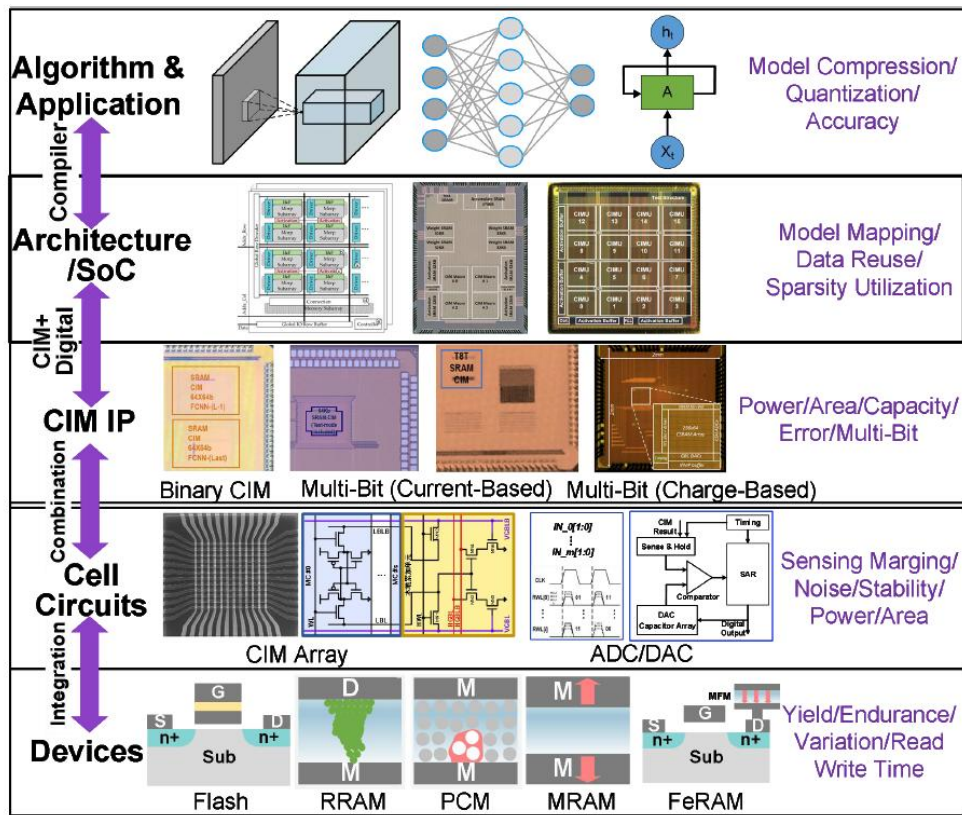
$$\begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_N \end{bmatrix}^T \cdot \begin{bmatrix} W_{11}, \dots, W_{1M} \\ W_{21}, \dots, W_{2M} \\ \dots \\ W_{N1}, \dots, W_{NM} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_N \end{bmatrix}$$

Large W , small X/Y to minimize data access

Ideal application: matrix-vector multiplication (FC)



/CIM Design Hierarchy

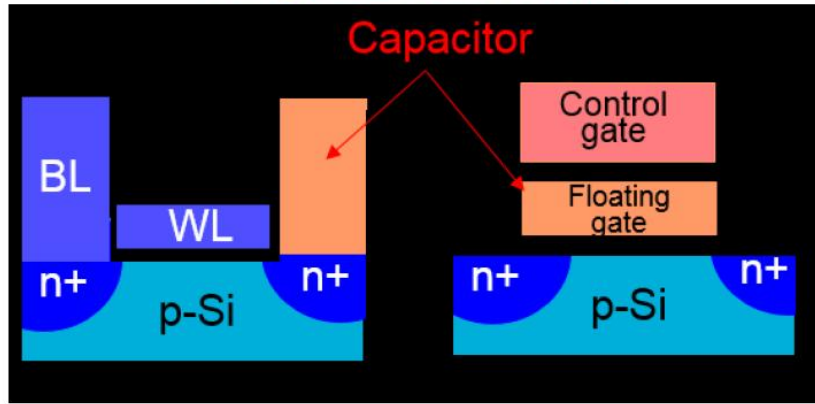


/Outlines

- **Introduction**
- **Memory devices and computational primitives**
 - **Charge-based memory devices**
 - **Resistance-based memory devices**
- **SRAM-based CIM: from device to architecture**
 - **3 SRAM CIM primitives**
 - **Toward efficient MVM mapping to CIM array**
 - **Sparsity in CIM**
- **Challenges, Trade-offs and Metrics of CIM**

/Constituent Elements

Charge-based memory



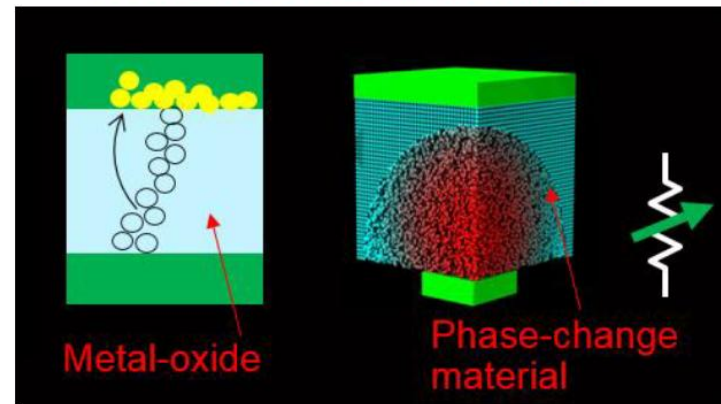
Coulomb's law

Aga et al., HPCA (2017) (SRAM)

Seshadri et al., MICRO (2017) (DRAM)

Merrikh-Bayat, IEEE TNNLS (2018) (Flash)

Resistance-based memory



Ohm's law

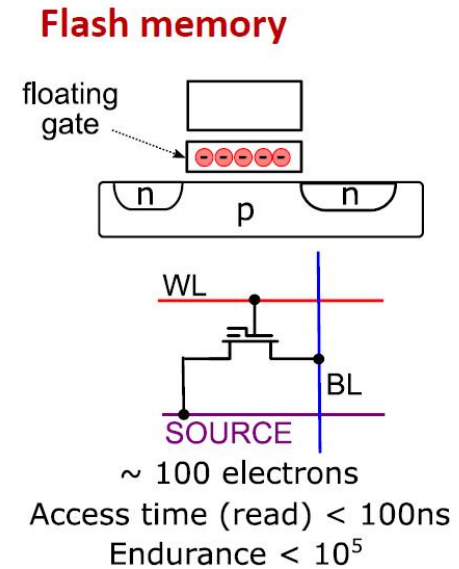
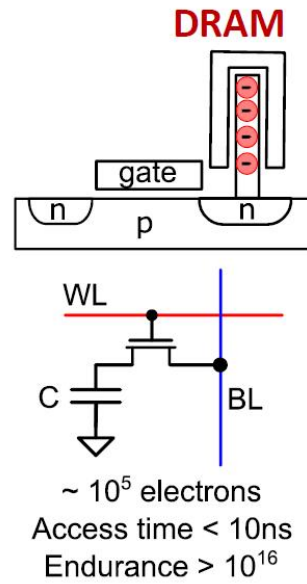
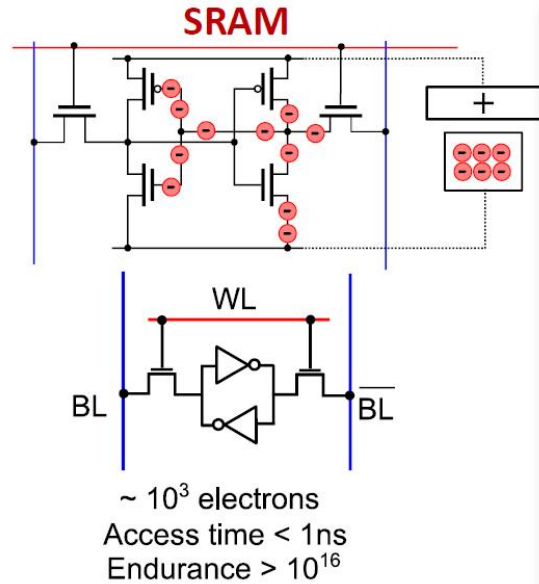
Burr et al., Adv. Phys. X (2017)

Sebastian et al., J. Appl. Phys. (2018)

Ielmini and Wong, Nature Electr. (2018)

- **Charge-based memory:** Presence or absence of charge (eg. DRAM, SRAM, Flash)
- **Resistance-based memory:** Differences in atomic arrangements or orientation of ferromagnetic metal layers (eg. PCM, metal-oxide RRAM, STT-MRAM)
- Several **computational primitives realized by both types of memory**

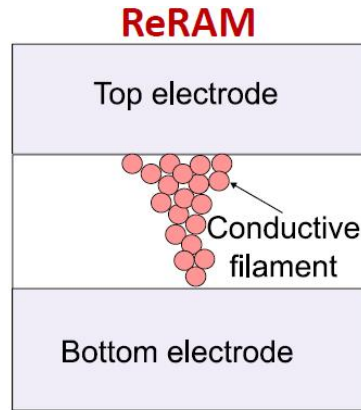
/ Charge-Based Memory Devices



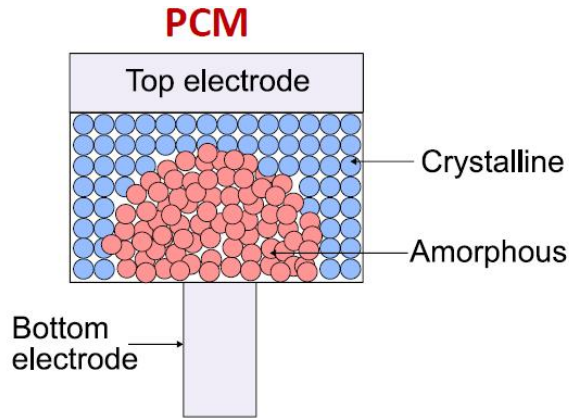
- **SRAM:** Two CMOS inverters connected back to back. The charge is confined within the barriers formed by FET channels and by gate insulators
- **DRAM:** Capacitor connected in series to a FET
- **Flash:** The storage node is coupled to the gate of a FET

*Emerging nanoelectronics
devices, John Wiley & Sons (2015)*

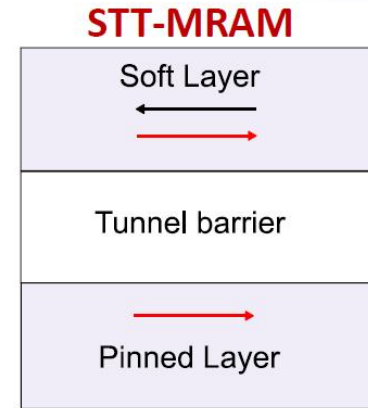
/Resistance-Based Memory Devices



Resistance range = 10^3 - 10^7
Access time (write) = 10ns - 100ns
Endurance = 10^6 - 10^9



Resistance range = 10^4 - 10^7
Access time (write) ~ 100ns
Endurance = 10^6 - 10^9



Resistance range = 10^3 - 10^4
Access time (write) < 10ns
Endurance > 10^{14}

- **ReRAM:** Migration of defects such as oxygen vacancies or metallic ions
- **PCM:** Joule-heating induced reversible phase transition
- **STT-MRAM:** Magnetic polarization of a free layer with respect to a pinned layer
- Resistance-based memory devices also referred to as **memristive devices**

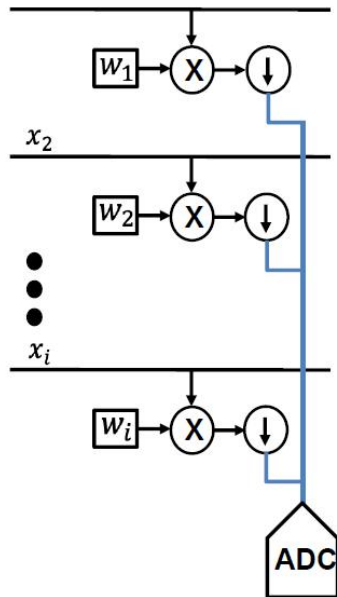
Wong and Salahuddin, Nature Nanotechnology (2015)

/Outlines

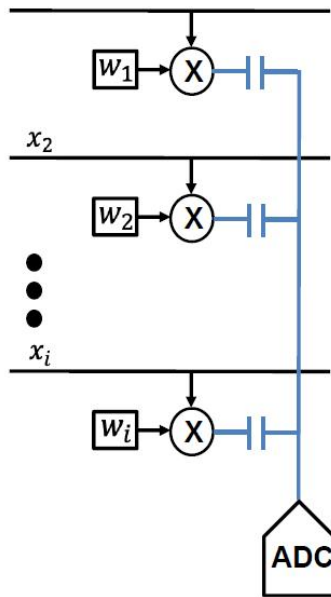
- **Introduction**
- **Memory devices and computational primitives**
 - **Charge-based memory devices**
 - **Resistance-based memory devices**
- **SRAM-based CIM: from device to architecture**
 - **3 SRAM CIM primitives**
 - **Toward efficient MVM mapping to CIM array**
 - **Sparsity in CIM**
- **Challenges, Trade-offs and Metrics of CIM**

Three Types of SRAM-based CIM

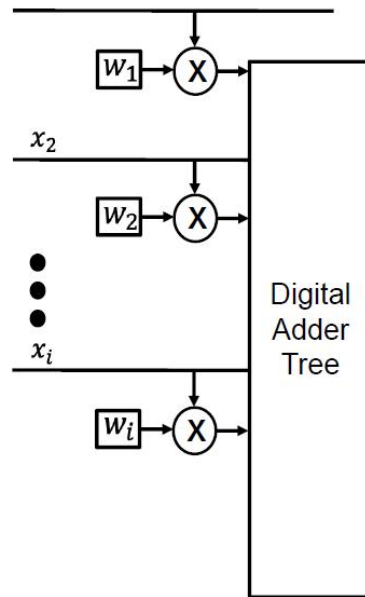
Classified by: Result generation format on each single CIM cell



Current-based CIM
(Less transistors)



Charge-based CIM
(High parallelism)



All-digital CIM
(No accuracy loss)

/SRAM CIM type 1: current-based

□ Computing-in memory basic operation

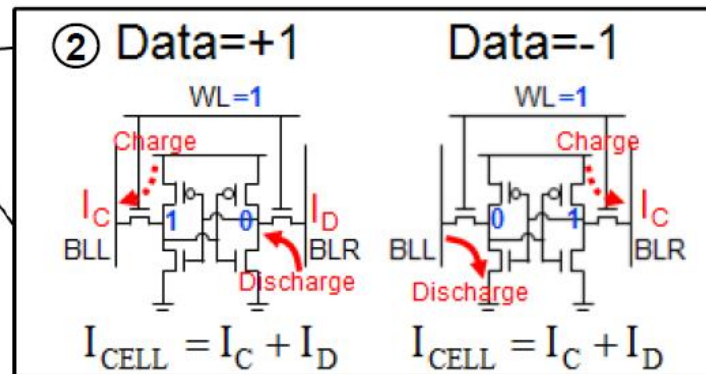
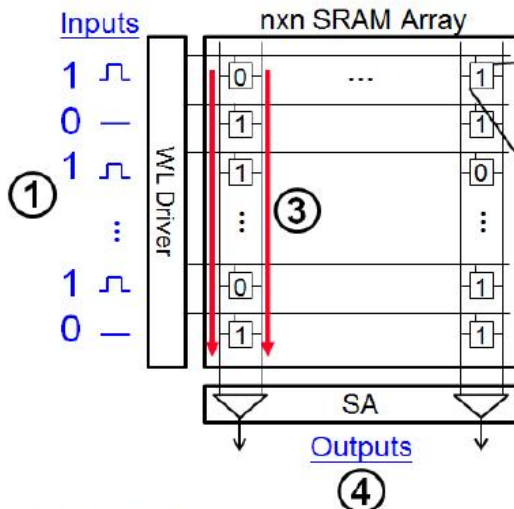
Input/WL (IN)	Weight (W)	Product (INxW)	I_{MC} on BLL	I_{MC} on BLR
0	X	0	0	0
1	+1	+1	I_C	I_D
1	-1	-1	I_D	I_C

(1) Input as wordline signals

(2) If WL=1, SRAM conducts currents

(3) Current accumulation

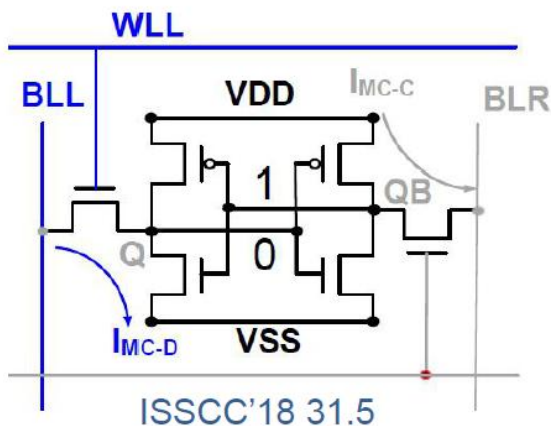
(4) SA comparison



* By design, $I_D \gg I_C$

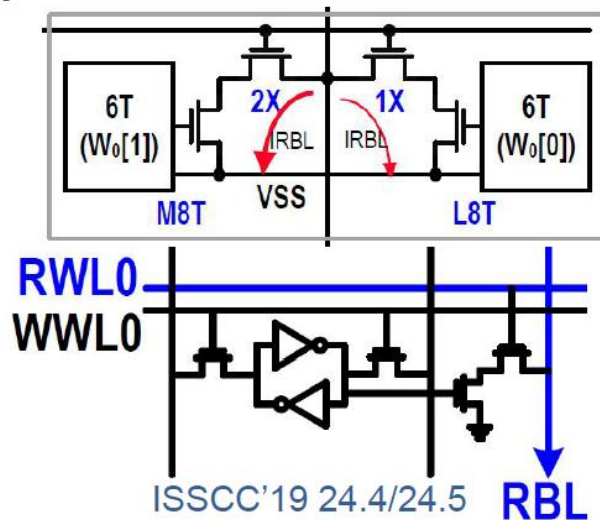
/SRAM CIM type 1: current-based

- Stability (write-disturb)
- Increase sensing margin
- Reduce PVT variation
- Reduce cell area/power



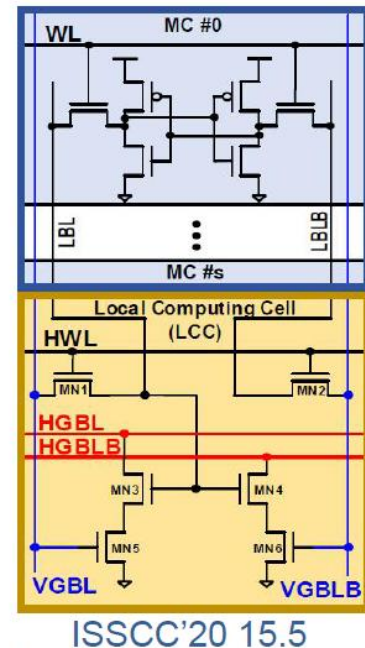
6T CIM cell

Small area/Write disturb/Sensing margin



8T CIM Cell

Sensing margin/Larger area

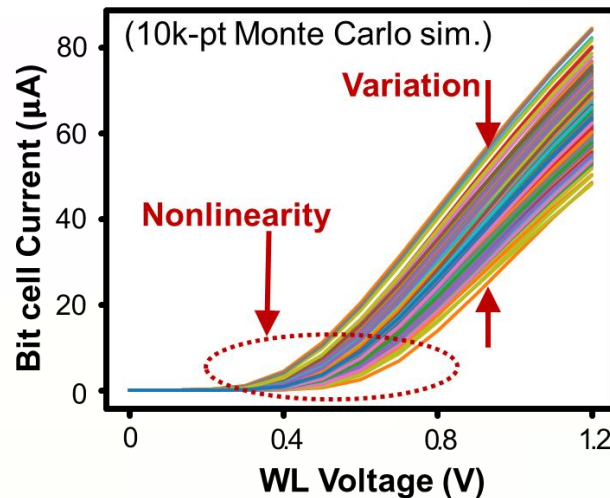
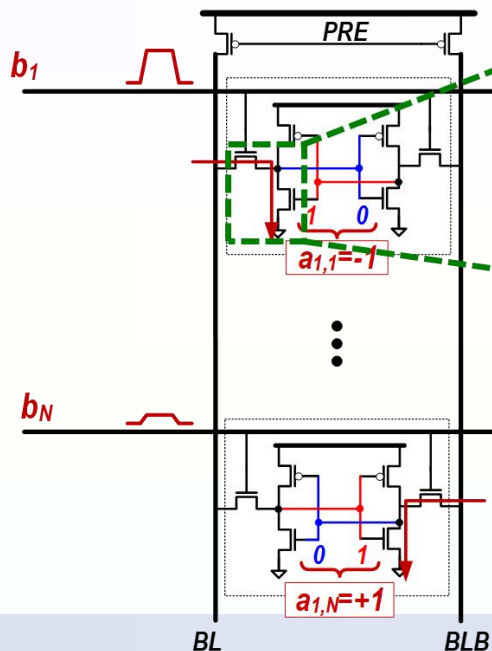


Block-wise 6T CIM cell

Sensing margin/Moderate area

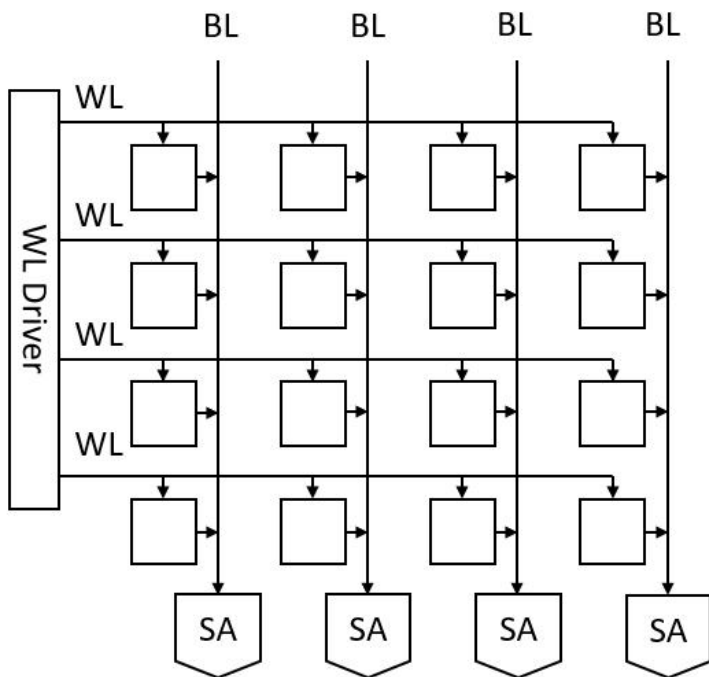
SRAM CIM type 1: current-based

Current-domain SRAM:



- Use analog circuits to 'fit' computation in bit cells
 - Compute SNR limited by circuit non-idealities (nonlinearity, variation, temperature)

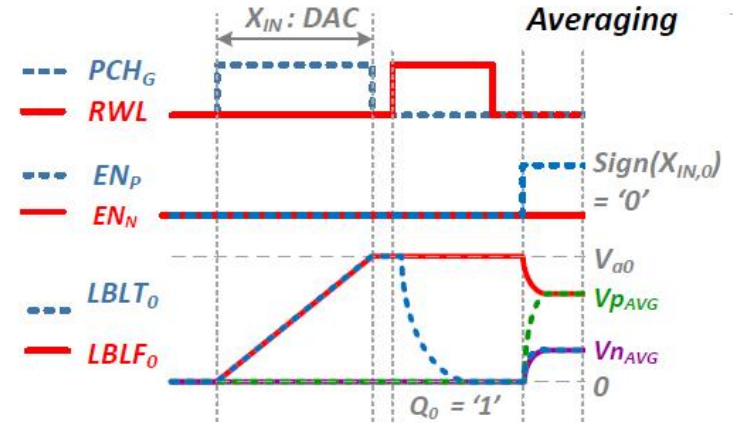
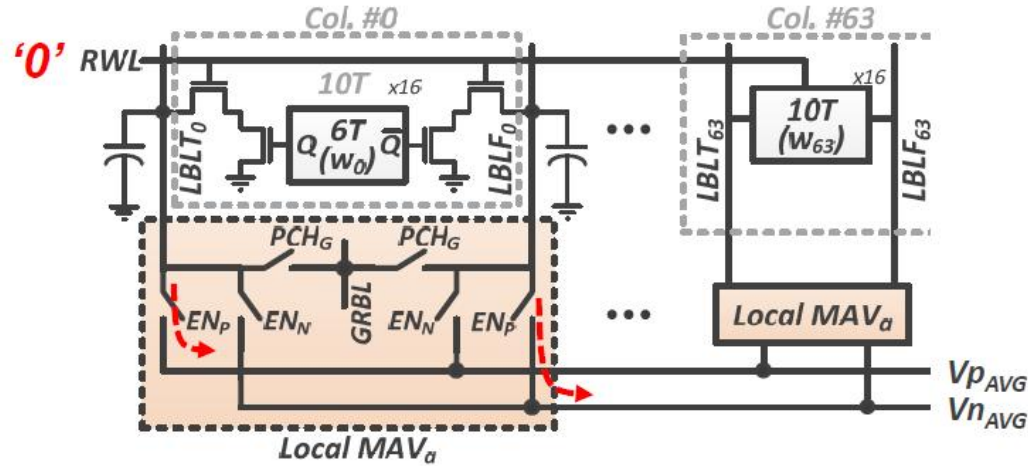
/ SRAM CIM type 1: current-based



输入: wordline

输出: bitline 电流累加

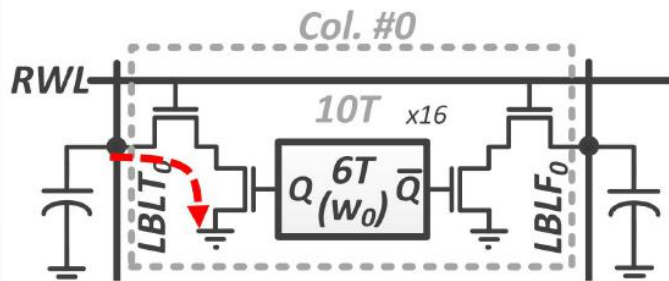
/ SRAM CIM type 2: charge-based



- Charge-sharing of local bit-lines implements averaging
- Separate voltage rails for '+ve' ($V_{p_{AVG}}$) and '-ve' ($V_{n_{AVG}}$) parts of the average

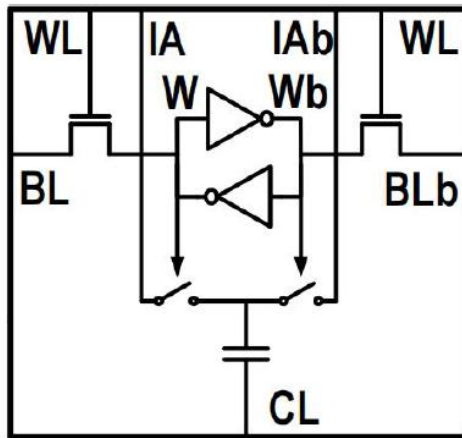
/SRAM CIM type 2: charge-based

- Increase sensing margin
- Reduce PVT variation
- Reduce cell area/power



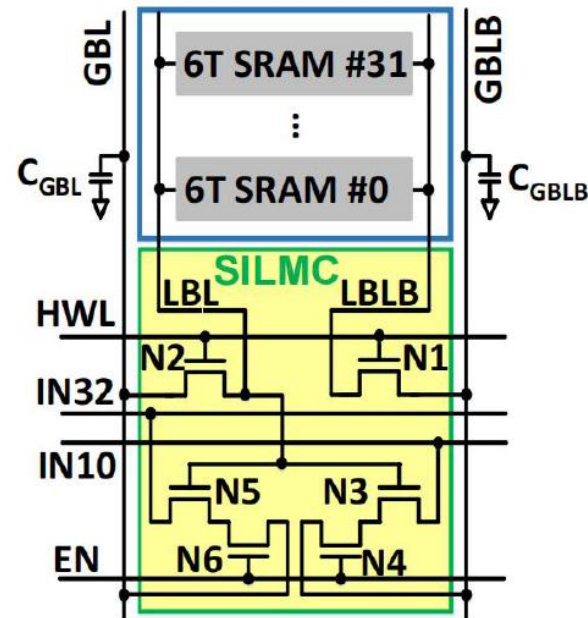
ISSCC'18 31.1

10T CIM cell



ISSCC'21 15.1

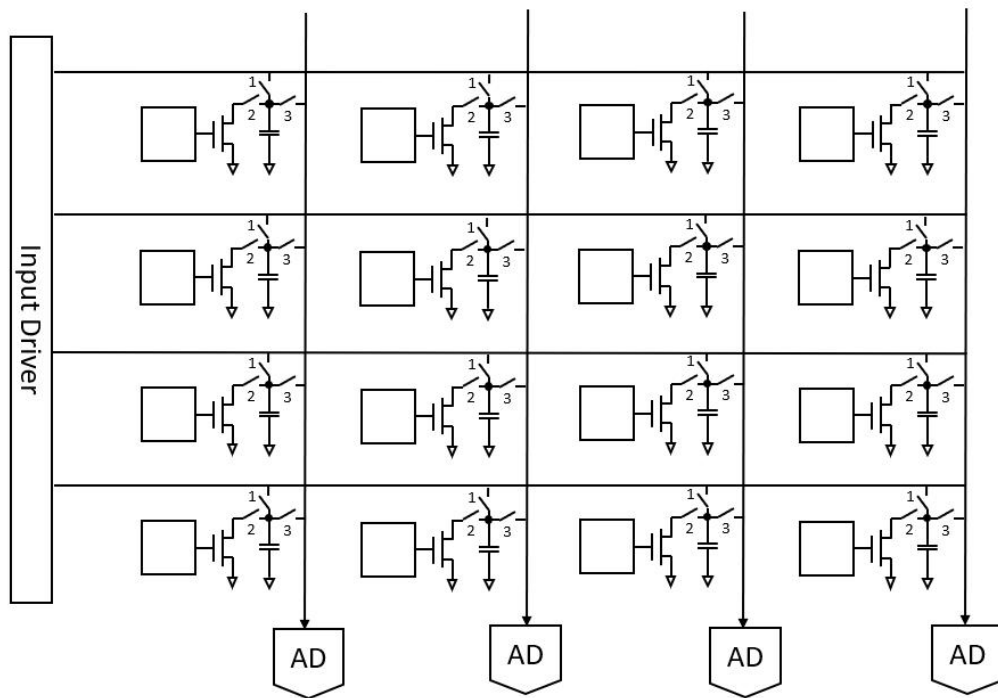
8T CIM Cell



ISSCC'21 16.3

Block-wise 6T CIM cell

/ SRAM CIM type 2: charge-based



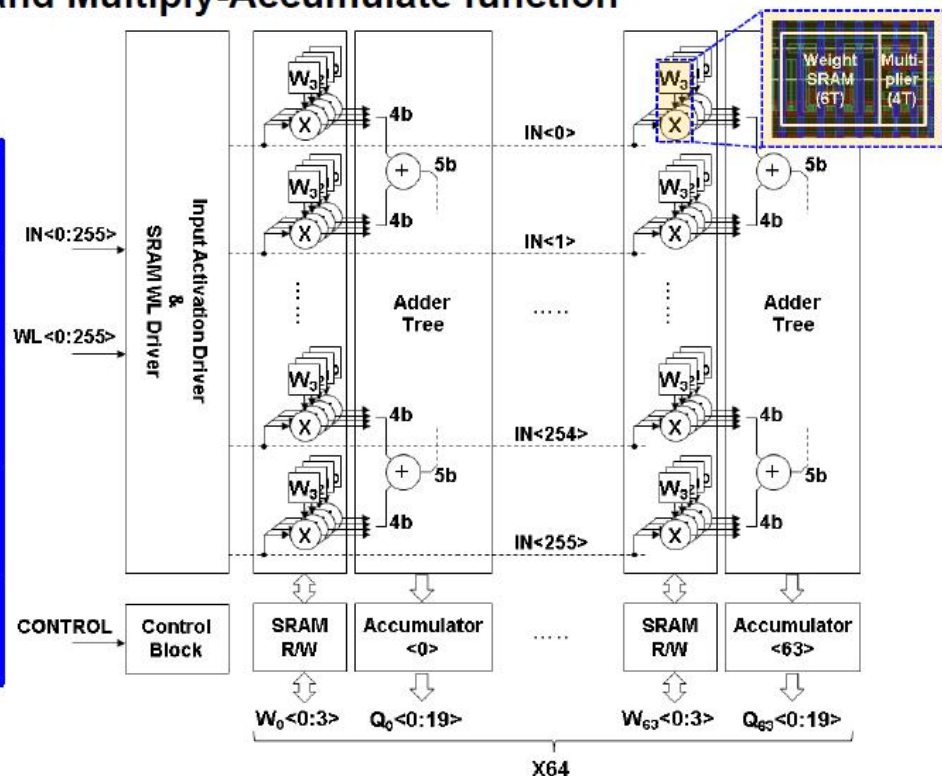
1. 电容预充至输入值
2. 存储0值的cell对应电容放电
3. 电容并联，直接计算电压

type2: charge-based

/SRAM CIM type 3: all-digital

■ A Macro to support Weight storage and Multiply-Accumulate function

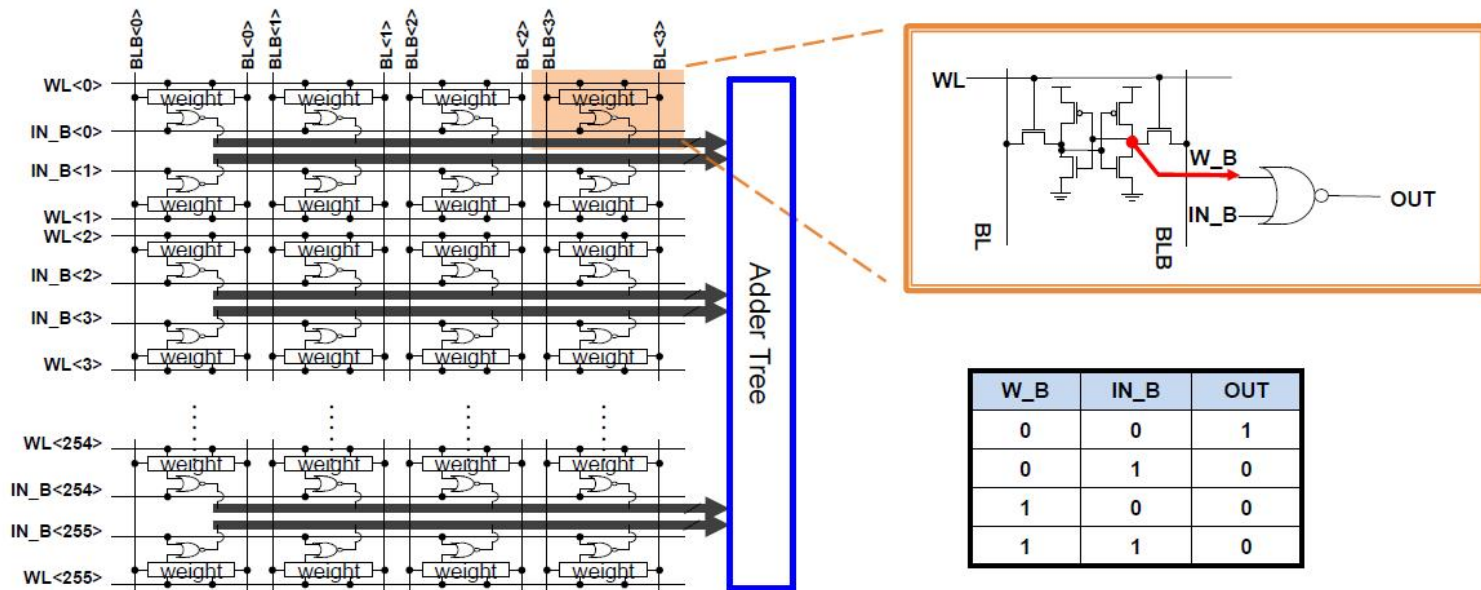
Main building blocks	Weight Update/Storage	Multiply Accumulate
Array	6T SRAM for weight storage	Multiplier Adder
Y-pitch Circuit	Row decoder Row driver	Input mux/driver
X-pitch Circuit	Sense amplifier Write driver	Accumulator
Control	Timing control Address decode	MAC control logic/timing



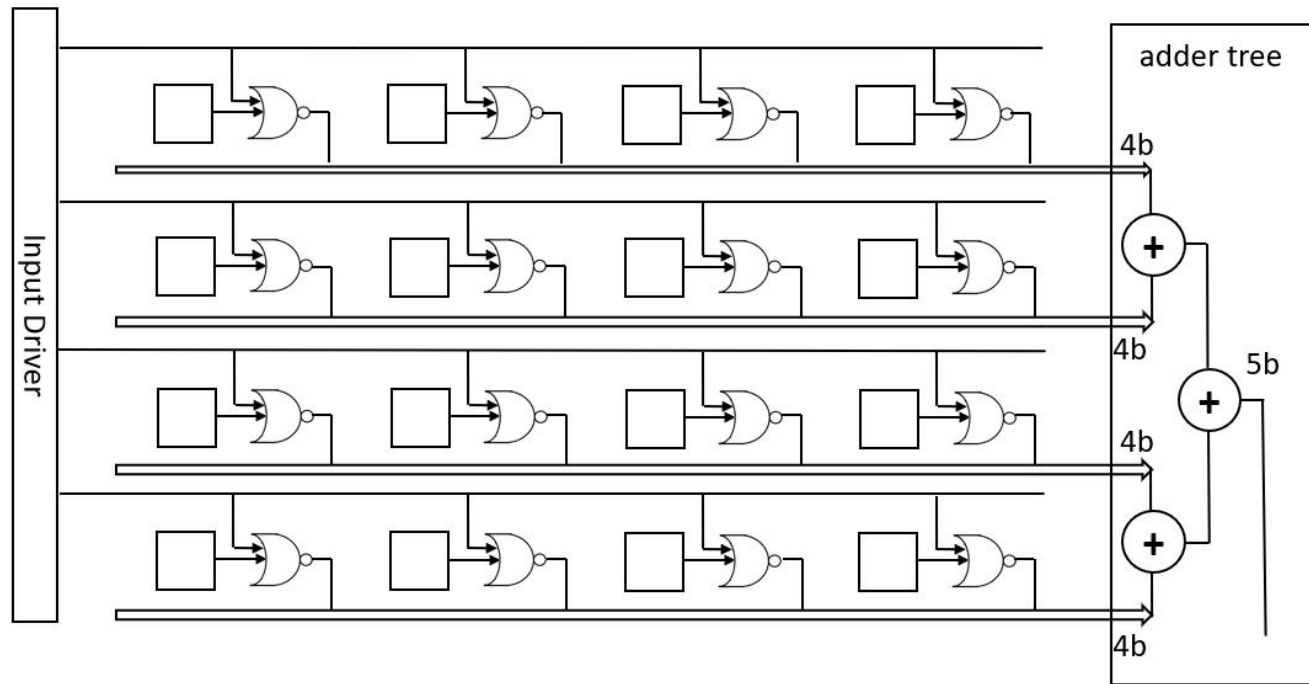
X64

SRAM CIM type 3: all-digital

- 6T SRAM for weight storage , NOR gate for multiply
- Weight access through WL and BL/BLB
- SRAM can be at low-voltage retention mode during MAC operation



/ SRAM CIM type 3: all-digital



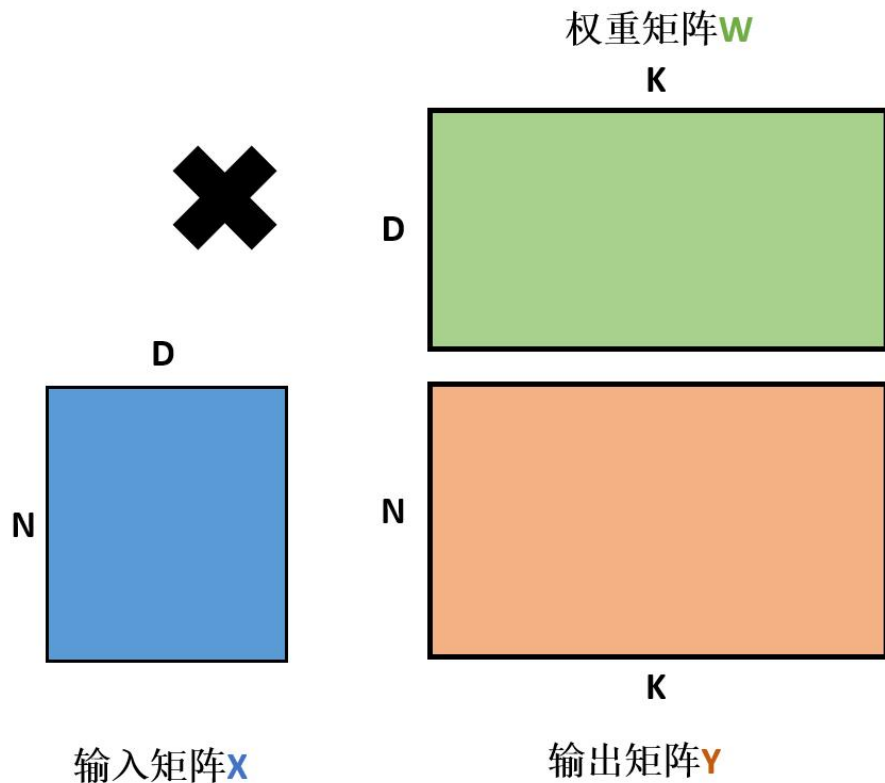
使用NOR gate,
实际输入/存储: 取反
后的input/weight。

type3: all-digital

/Outlines

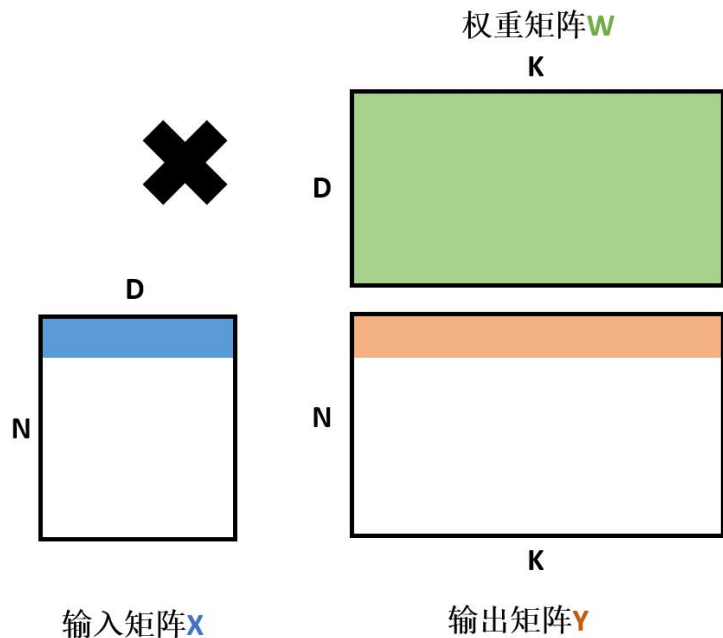
- **Introduction**
- **Memory devices and computational primitives**
 - **Charge-based memory devices**
 - **Resistance-based memory devices**
- **SRAM-based CIM: from device to architecture**
 - **3 SRAM CIM primitives**
 - **Toward efficient MVM mapping to CIM array**
 - **Sparsity in CIM**
- **Challenges, Trade-offs and Metrics of CIM**

/ 矩阵乘运算



```
for (n=0;n<N;n++)  
  for (k=0;k<K;k++)  
    Y[n,k] = 0;  
    for (d=0;d<D;d++)  
      Y[n,k] += X[n,d] * W[d,k];
```

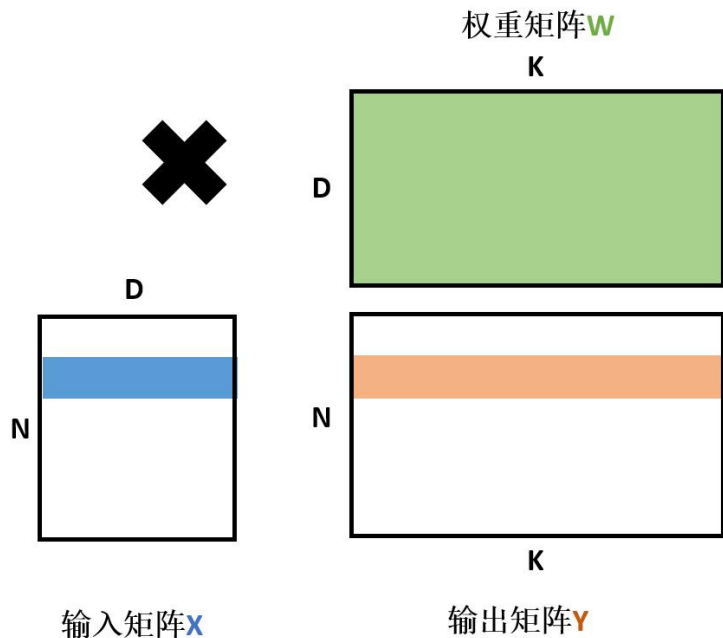
矩阵乘运算



```
for (n=0;n<N;n++)  
  for (k=0;k<K;k++)  
    Y[n,k] = 0;  
    for (d=0;d<D;d++)  
      Y[n,k] += X[n,d] * W[d,k];
```

$x[0,0]$	$\begin{matrix} W \\ [0,0] \end{matrix}$	$\begin{matrix} W \\ [0,1] \end{matrix}$	$\begin{matrix} W \\ [0,2] \end{matrix}$	$\begin{matrix} W \\ [0,3] \end{matrix}$
$x[0,1]$	$\begin{matrix} W \\ [1,0] \end{matrix}$	$\begin{matrix} W \\ [1,1] \end{matrix}$	$\begin{matrix} W \\ [1,2] \end{matrix}$	$\begin{matrix} W \\ [1,3] \end{matrix}$
$x[0,2]$	$\begin{matrix} W \\ [2,0] \end{matrix}$	$\begin{matrix} W \\ [2,1] \end{matrix}$	$\begin{matrix} W \\ [2,2] \end{matrix}$	$\begin{matrix} W \\ [2,3] \end{matrix}$
$x[0,3]$	$\begin{matrix} W \\ [3,0] \end{matrix}$	$\begin{matrix} W \\ [3,1] \end{matrix}$	$\begin{matrix} W \\ [3,2] \end{matrix}$	$\begin{matrix} W \\ [3,3] \end{matrix}$
	$Y[0,0]$	$Y[0,1]$	$Y[0,2]$	$Y[0,3]$

矩阵乘运算

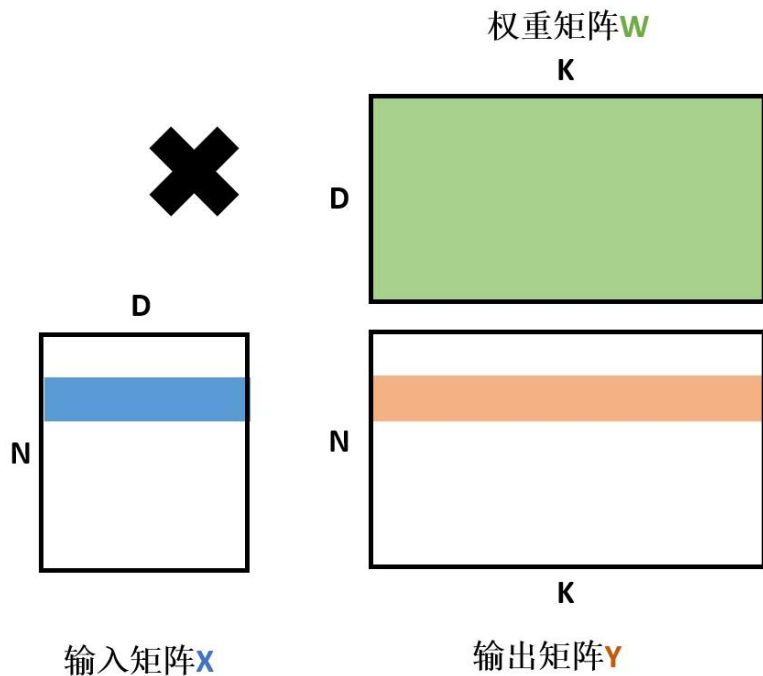


```
for (n=0;n<N;n++)  
  for (k=0;k<K;k++)  
    Y[n,k] = 0;  
    for (d=0;d<D;d++)  
      Y[n,k] += X[n,d] * W[d,k];
```

$X[1,0]$	$\begin{matrix} W \\ [0,0] \end{matrix}$	$\begin{matrix} W \\ [0,1] \end{matrix}$	$\begin{matrix} W \\ [0,2] \end{matrix}$	$\begin{matrix} W \\ [0,3] \end{matrix}$
$X[1,1]$	$\begin{matrix} W \\ [1,0] \end{matrix}$	$\begin{matrix} W \\ [1,1] \end{matrix}$	$\begin{matrix} W \\ [1,2] \end{matrix}$	$\begin{matrix} W \\ [1,3] \end{matrix}$
$X[1,2]$	$\begin{matrix} W \\ [2,0] \end{matrix}$	$\begin{matrix} W \\ [2,1] \end{matrix}$	$\begin{matrix} W \\ [2,2] \end{matrix}$	$\begin{matrix} W \\ [2,3] \end{matrix}$
$X[1,3]$	$\begin{matrix} W \\ [3,0] \end{matrix}$	$\begin{matrix} W \\ [3,1] \end{matrix}$	$\begin{matrix} W \\ [3,2] \end{matrix}$	$\begin{matrix} W \\ [3,3] \end{matrix}$
	$Y[1,0]$	$Y[1,1]$	$Y[1,2]$	$Y[1,3]$

WS (weight stationary), X 按行输入, Y 按行输出

Spatial_for: 并行计算优化

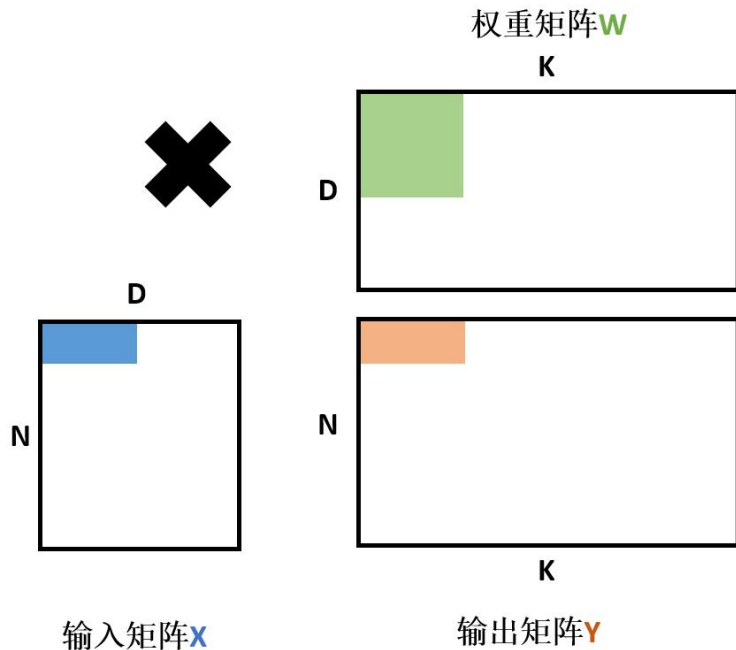


```
for (n=0;n<N;n++)  
    spatial_for (k=0;k<K;k++) //column  
        Y[n,k] = 0;  
    spatial_for (d=0;d<D;d++) //cell  
        Y[n,k] += X[n,d]* W[d,k];
```

$X[1,0]$	$\begin{matrix} W \\ [0,0] \end{matrix}$	$\begin{matrix} W \\ [0,1] \end{matrix}$	$\begin{matrix} W \\ [0,2] \end{matrix}$	$\begin{matrix} W \\ [0,3] \end{matrix}$
$X[1,1]$	$\begin{matrix} W \\ [1,0] \end{matrix}$	$\begin{matrix} W \\ [1,1] \end{matrix}$	$\begin{matrix} W \\ [1,2] \end{matrix}$	$\begin{matrix} W \\ [1,3] \end{matrix}$
$X[1,2]$	$\begin{matrix} W \\ [2,0] \end{matrix}$	$\begin{matrix} W \\ [2,1] \end{matrix}$	$\begin{matrix} W \\ [2,2] \end{matrix}$	$\begin{matrix} W \\ [2,3] \end{matrix}$
$X[1,3]$	$\begin{matrix} W \\ [3,0] \end{matrix}$	$\begin{matrix} W \\ [3,1] \end{matrix}$	$\begin{matrix} W \\ [3,2] \end{matrix}$	$\begin{matrix} W \\ [3,3] \end{matrix}$
	$Y[1,0]$	$Y[1,1]$	$Y[1,2]$	$Y[1,3]$

Q1: What if the systolic array size $< K$ or D ?

A: 每次写入 $D_0 \times K_0$ 个权重计算，输入和输出相应进行tiling。实际上这是OS (output stationary) 数据流。



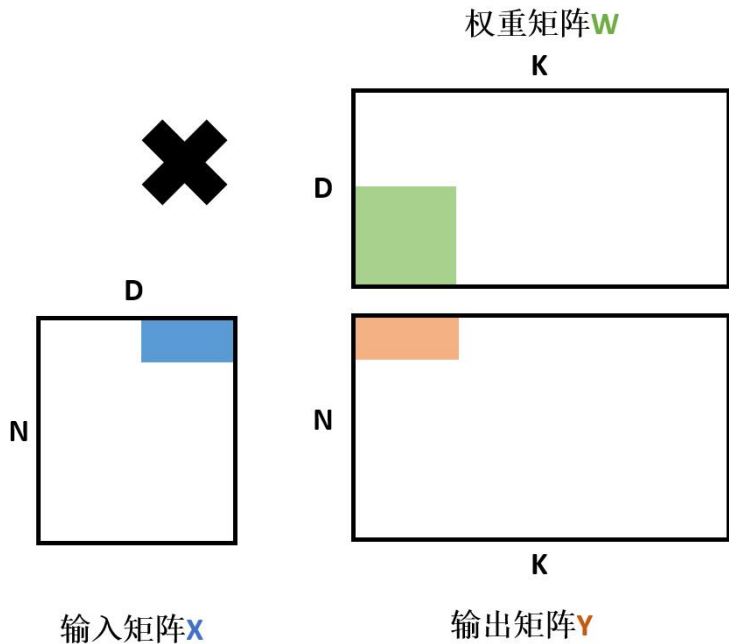
```
for (n=0;n<N;n++)
  for (k1=0;k1<K1;k1++)
     $Y[n, k1 \times K_0 : (k1+1) \times K_0] = 0;$ 
    for (d1=0;d1<D1;d1++)
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
           $Y[n, k1 \times K_0 + k_0] += X[n, d1 \times D_0 + d_0] * W[d1 \times D_0 + d_0, k1 \times K_0 + k_0];$ 
```

$x[0,0]$	$\begin{matrix} W \\ [0,0] \end{matrix}$	$\begin{matrix} W \\ [0,1] \end{matrix}$	$\begin{matrix} W \\ [0,2] \end{matrix}$	$\begin{matrix} W \\ [0,3] \end{matrix}$
$x[0,1]$	$\begin{matrix} W \\ [1,0] \end{matrix}$	$\begin{matrix} W \\ [1,1] \end{matrix}$	$\begin{matrix} W \\ [1,2] \end{matrix}$	$\begin{matrix} W \\ [1,3] \end{matrix}$
$x[0,2]$	$\begin{matrix} W \\ [2,0] \end{matrix}$	$\begin{matrix} W \\ [2,1] \end{matrix}$	$\begin{matrix} W \\ [2,2] \end{matrix}$	$\begin{matrix} W \\ [2,3] \end{matrix}$
$x[0,3]$	$\begin{matrix} W \\ [3,0] \end{matrix}$	$\begin{matrix} W \\ [3,1] \end{matrix}$	$\begin{matrix} W \\ [3,2] \end{matrix}$	$\begin{matrix} W \\ [3,3] \end{matrix}$

$Y[0,0] \ Y[0,1] \ Y[0,2] \ Y[0,3]$

Q1: What if the systolic array size $< K$ or D ?

A: 每次写入 $D_0 \times K_0$ 个权重计算，输入和输出相应进行tiling。实际上这是OS (output stationary) 数据流。



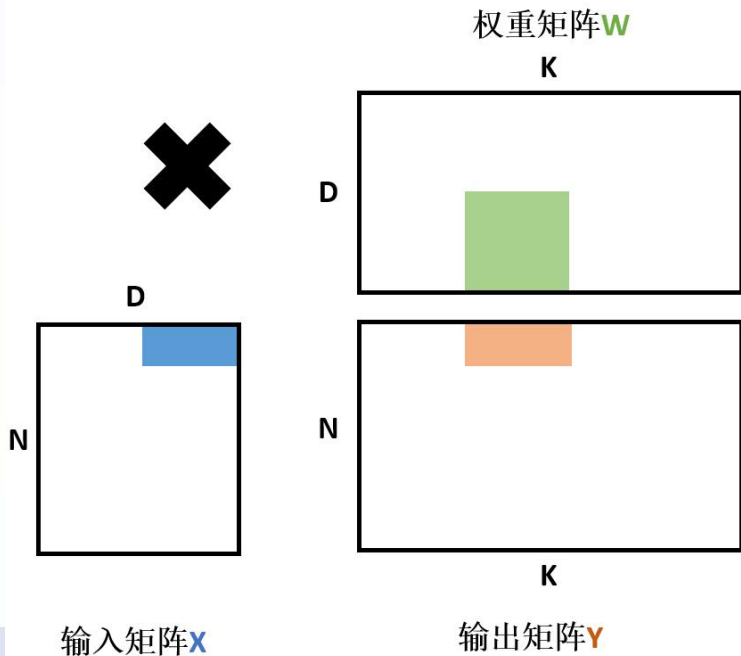
```
for (n=0;n<N;n++)
  for (k1=0;k1<K1;k1++)
    Y[n,k1*K0:(k1+1)*K0] = 0;
    for (d1=0;d1<D1;d1++)
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
          Y[n,k1*K0+k0] += X[n,d1*D0+d0]*
                           W[d1*D0+d0,k1*K0+k0];
```

$x[0,4]$	$\begin{matrix} W \\ [4,0] \end{matrix}$	$\begin{matrix} W \\ [4,1] \end{matrix}$	$\begin{matrix} W \\ [4,2] \end{matrix}$	$\begin{matrix} W \\ [4,3] \end{matrix}$
$x[0,5]$	$\begin{matrix} W \\ [5,0] \end{matrix}$	$\begin{matrix} W \\ [5,1] \end{matrix}$	$\begin{matrix} W \\ [5,2] \end{matrix}$	$\begin{matrix} W \\ [5,3] \end{matrix}$
$x[0,6]$	$\begin{matrix} W \\ [6,0] \end{matrix}$	$\begin{matrix} W \\ [6,1] \end{matrix}$	$\begin{matrix} W \\ [6,2] \end{matrix}$	$\begin{matrix} W \\ [6,3] \end{matrix}$
$x[0,7]$	$\begin{matrix} W \\ [7,0] \end{matrix}$	$\begin{matrix} W \\ [7,1] \end{matrix}$	$\begin{matrix} W \\ [7,2] \end{matrix}$	$\begin{matrix} W \\ [7,3] \end{matrix}$

$Y[0,0] \ Y[0,1] \ Y[0,2] \ Y[0,3]$

Q1: What if the systolic array size < K or D?

A: 每次写入 $D_0 \times K_0$ 个权重计算，输入和输出相应进行tiling。实际上这是OS (output stationary) 数据流。



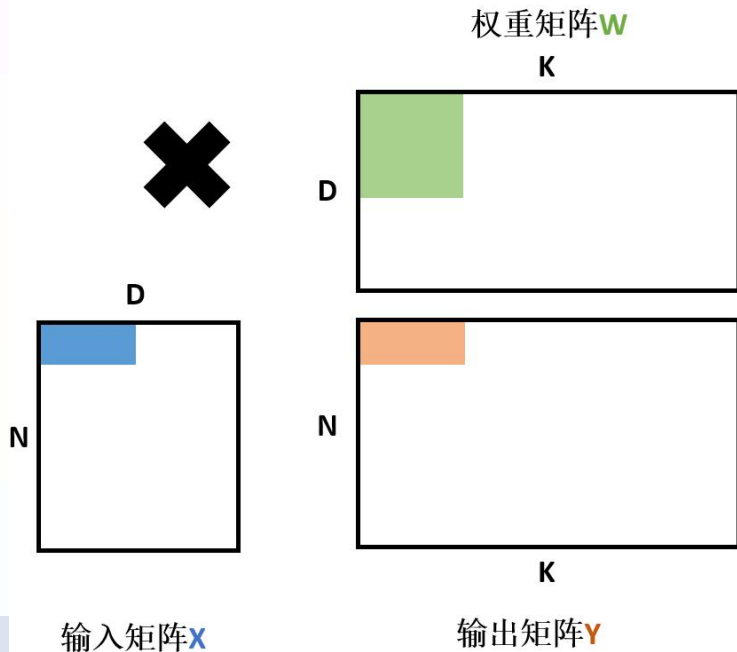
```
for (n=0;n<N;n++)
  for (k1=0;k1<K1;k1++)
    Y[n,k1*K0:(k1+1)*K0] = 0;
    for (d1=0;d1<D1;d1++)
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
          Y[n,k1*K0+k0] += X[n,d1*D0+d0]*
            W[d1*D0+d0,k1*K0+k0];
```

$x[0,4]$	$\begin{matrix} W \\ [4,4] \end{matrix}$	$\begin{matrix} W \\ [4,5] \end{matrix}$	$\begin{matrix} W \\ [4,6] \end{matrix}$	$\begin{matrix} W \\ [4,7] \end{matrix}$
$x[0,5]$	$\begin{matrix} W \\ [5,4] \end{matrix}$	$\begin{matrix} W \\ [5,5] \end{matrix}$	$\begin{matrix} W \\ [5,6] \end{matrix}$	$\begin{matrix} W \\ [5,7] \end{matrix}$
$x[0,6]$	$\begin{matrix} W \\ [6,4] \end{matrix}$	$\begin{matrix} W \\ [6,5] \end{matrix}$	$\begin{matrix} W \\ [6,6] \end{matrix}$	$\begin{matrix} W \\ [6,7] \end{matrix}$
$x[0,7]$	$\begin{matrix} W \\ [7,4] \end{matrix}$	$\begin{matrix} W \\ [7,5] \end{matrix}$	$\begin{matrix} W \\ [7,6] \end{matrix}$	$\begin{matrix} W \\ [7,7] \end{matrix}$

$Y[0,4]$ $Y[0,5]$ $Y[0,6]$ $Y[0,7]$

Q2: 对于CIM我们希望减少weight的移动, 这种方法移动的太多, 如何解决?

A: 调整循环顺序, 减少weight移动。实际上这是WS (weight stationary) 数据流。



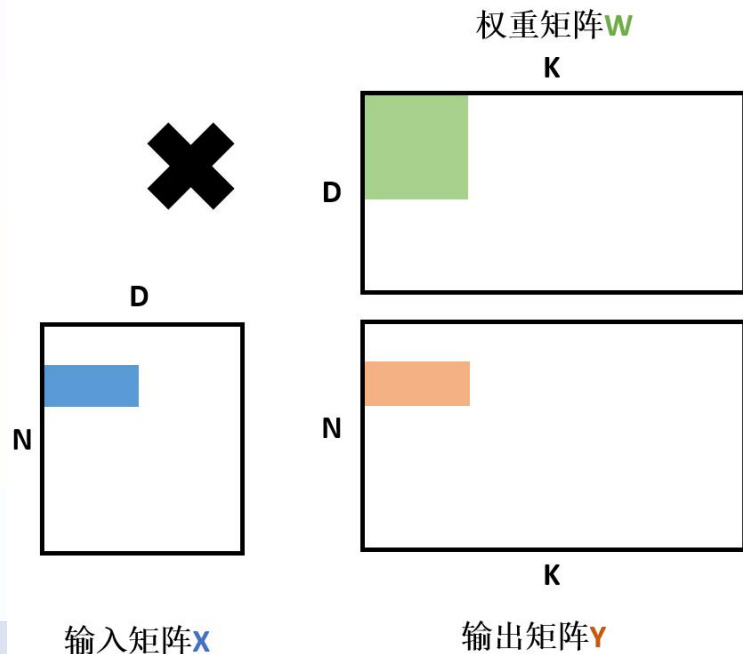
```
for (k1=0;k1<K1;k1++)
  for (d1=0;d1<D1;d1++)
    for (n=0;n<N;n++)
      if (d1==0) Y[n,k1*K0:(k1+1)*K0] = 0;
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
          Y[n,k1*K0+k0] += X[n,d1*D0+d0]*
                           W[d1*D0+d0,k1*K0+k0];
```

$x[0,0]$	$\begin{matrix} W \\ [0,0] \end{matrix}$	$\begin{matrix} W \\ [0,1] \end{matrix}$	$\begin{matrix} W \\ [0,2] \end{matrix}$	$\begin{matrix} W \\ [0,3] \end{matrix}$
$x[0,1]$	$\begin{matrix} W \\ [1,0] \end{matrix}$	$\begin{matrix} W \\ [1,1] \end{matrix}$	$\begin{matrix} W \\ [1,2] \end{matrix}$	$\begin{matrix} W \\ [1,3] \end{matrix}$
$x[0,2]$	$\begin{matrix} W \\ [2,0] \end{matrix}$	$\begin{matrix} W \\ [2,1] \end{matrix}$	$\begin{matrix} W \\ [2,2] \end{matrix}$	$\begin{matrix} W \\ [2,3] \end{matrix}$
$x[0,3]$	$\begin{matrix} W \\ [3,0] \end{matrix}$	$\begin{matrix} W \\ [3,1] \end{matrix}$	$\begin{matrix} W \\ [3,2] \end{matrix}$	$\begin{matrix} W \\ [3,3] \end{matrix}$

$Y[0,0]$ $Y[0,1]$ $Y[0,2]$ $Y[0,3]$

Q2: 对于CIM我们希望减少weight的移动, 这种方法移动的太多, 如何解决?

A: 调整循环顺序, 减少weight移动。实际上这是WS (weight stationary) 数据流。



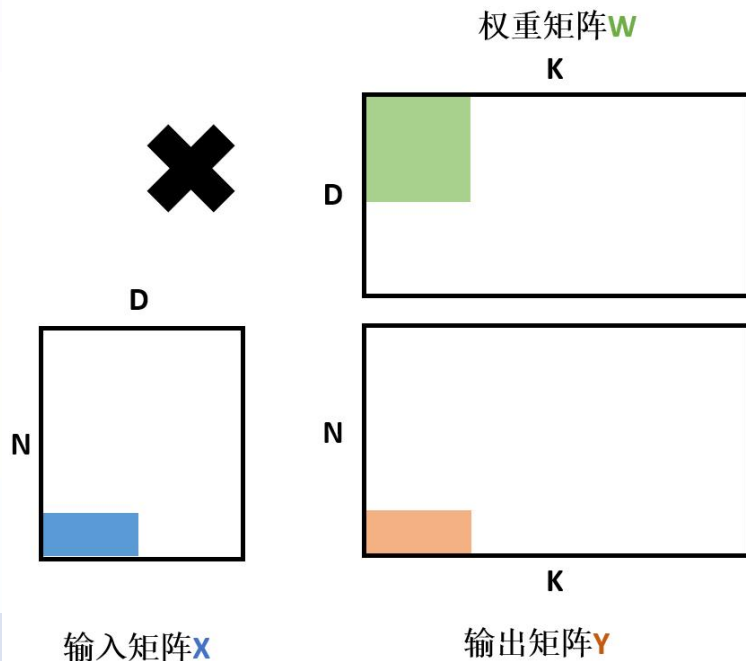
```
for (k1=0;k1<K1;k1++)
  for (d1=0;d1<D1;d1++)
    for (n=0;n<N;n++)
      if (d1==0) Y[n,k1*K0:(k1+1)*K0] = 0;
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
          Y[n,k1*K0+k0] += X[n,d1*D0+d0]*
            W[d1*D0+d0,k1*K0+k0];
```

x[1,0]	W [0,0]	W [0,1]	W [0,2]	W [0,3]
x[1,1]	W [1,0]	W [1,1]	W [1,2]	W [1,3]
x[1,2]	W [2,0]	W [2,1]	W [2,2]	W [2,3]
x[1,3]	W [3,0]	W [3,1]	W [3,2]	W [3,3]

Y[1,0] Y[1,1] Y[1,2] Y[1,3]

Q2: 对于CIM我们希望减少weight的移动, 这种方法移动的太多, 如何解决?

A: 调整循环顺序, 减少weight移动。实际上这是WS (weight stationary) 数据流。



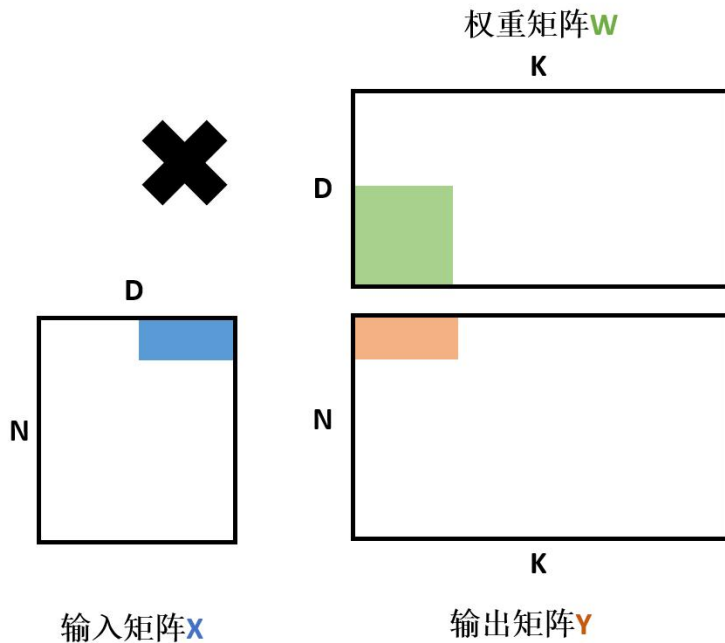
```
for (k1=0;k1<K1;k1++)  
  for (d1=0;d1<D1;d1++)  
    for (n=0;n<N;n++)  
      if (d1==0)  $Y[n,k1*K0:(k1+1)*K0] = 0;$   
      spatial_for (k0=0;k0<K0;k0++) //column  
        spatial_for (d0=0;d0<D0;d0++) //cell  
           $Y[n,k1*K0+k0] += X[n,d1*D0+d0]*$   
             $W[d1*D0+d0,k1*K0+k0];$ 
```

$X[N,0]$	$\begin{bmatrix} W \\ [0,0] \end{bmatrix}$	$\begin{bmatrix} W \\ [0,1] \end{bmatrix}$	$\begin{bmatrix} W \\ [0,2] \end{bmatrix}$	$\begin{bmatrix} W \\ [0,3] \end{bmatrix}$
$X[N,1]$	$\begin{bmatrix} W \\ [1,0] \end{bmatrix}$	$\begin{bmatrix} W \\ [1,1] \end{bmatrix}$	$\begin{bmatrix} W \\ [1,2] \end{bmatrix}$	$\begin{bmatrix} W \\ [1,3] \end{bmatrix}$
$X[N,2]$	$\begin{bmatrix} W \\ [2,0] \end{bmatrix}$	$\begin{bmatrix} W \\ [2,1] \end{bmatrix}$	$\begin{bmatrix} W \\ [2,2] \end{bmatrix}$	$\begin{bmatrix} W \\ [2,3] \end{bmatrix}$
$X[N,3]$	$\begin{bmatrix} W \\ [3,0] \end{bmatrix}$	$\begin{bmatrix} W \\ [3,1] \end{bmatrix}$	$\begin{bmatrix} W \\ [3,2] \end{bmatrix}$	$\begin{bmatrix} W \\ [3,3] \end{bmatrix}$

$Y[N,0] \ Y[N,1] \ Y[N,2] \ Y[N,3]$

Q2: 对于CIM我们希望减少weight的移动，这种方法移动的太多，如何解决？

A: 调整循环顺序，减少weight移动。实际上这是WS (weight stationary) 数据流。



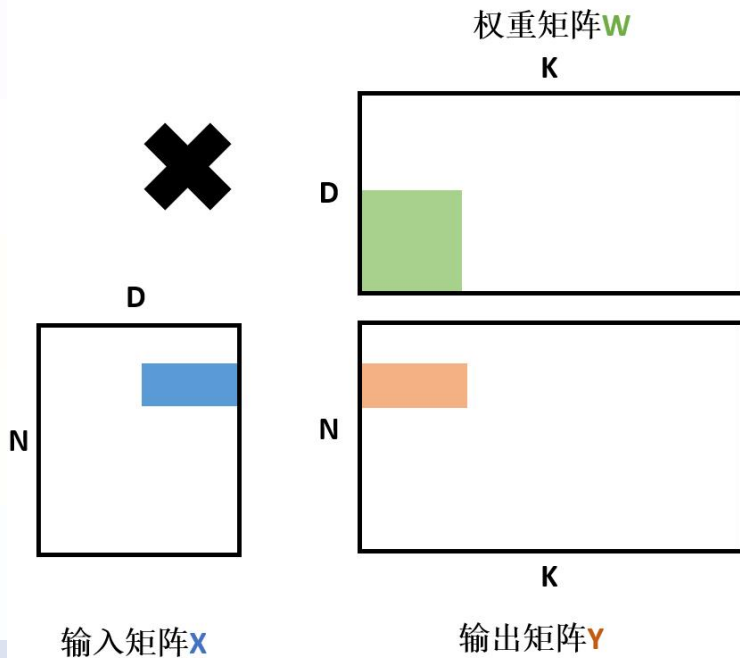
```
for (k1=0;k1<K1;k1++)
  for (d1=0;d1<D1;d1++)
    for (n=0;n<N;n++)
      if (d1==0)  $Y[n,k1*K0:(k1+1)*K0] = 0;$ 
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
           $Y[n,k1*K0+k0] += X[n,d1*D0+d0]*$ 
             $W[d1*D0+d0,k1*K0+k0];$ 
```

$X[0,4]$	$\begin{matrix} W \\ [4,0] \end{matrix}$	$\begin{matrix} W \\ [4,1] \end{matrix}$	$\begin{matrix} W \\ [4,2] \end{matrix}$	$\begin{matrix} W \\ [4,3] \end{matrix}$
$X[0,5]$	$\begin{matrix} W \\ [5,0] \end{matrix}$	$\begin{matrix} W \\ [5,1] \end{matrix}$	$\begin{matrix} W \\ [5,2] \end{matrix}$	$\begin{matrix} W \\ [5,3] \end{matrix}$
$X[0,6]$	$\begin{matrix} W \\ [6,0] \end{matrix}$	$\begin{matrix} W \\ [6,1] \end{matrix}$	$\begin{matrix} W \\ [6,2] \end{matrix}$	$\begin{matrix} W \\ [6,3] \end{matrix}$
$X[0,7]$	$\begin{matrix} W \\ [7,0] \end{matrix}$	$\begin{matrix} W \\ [7,1] \end{matrix}$	$\begin{matrix} W \\ [7,2] \end{matrix}$	$\begin{matrix} W \\ [7,3] \end{matrix}$

$Y[0,0] \ Y[0,1] \ Y[0,2] \ Y[0,3]$

Q2: 对于CIM我们希望减少weight的移动，这种方法移动的太多，如何解决？

A: 调整循环顺序，减少weight移动。实际上这是WS (weight stationary) 数据流。



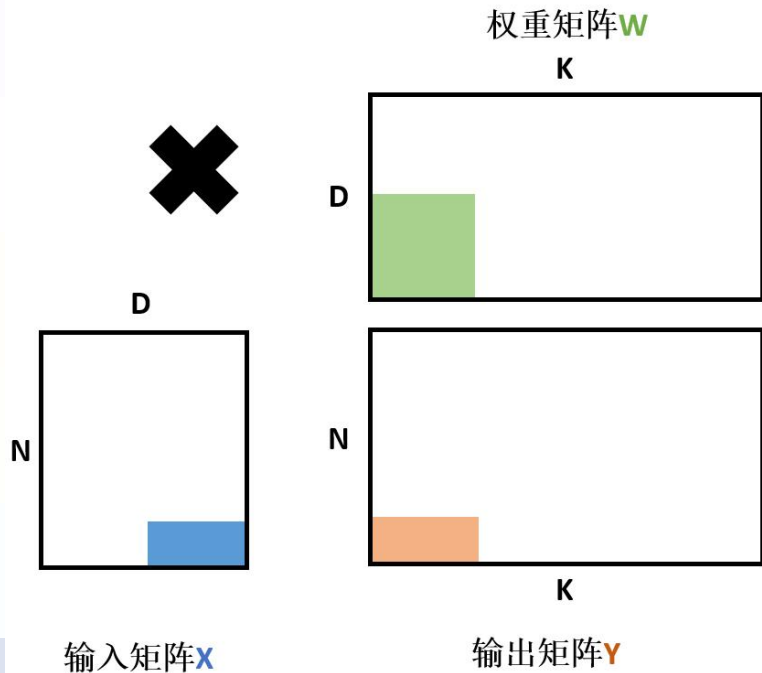
```
for (k1=0;k1<K1;k1++)
  for (d1=0;d1<D1;d1++)
    for (n=0;n<N;n++)
      if (d1==0)  $Y[n, k1 * K0 : (k1 + 1) * K0] = 0;$ 
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
           $Y[n, k1 * K0 + k0] += X[n, d1 * D0 + d0] * W[d1 * D0 + d0, k1 * K0 + k0];$ 
```

$X[4,4]$	$\begin{matrix} W \\ [4,0] \end{matrix}$	$\begin{matrix} W \\ [4,1] \end{matrix}$	$\begin{matrix} W \\ [4,2] \end{matrix}$	$\begin{matrix} W \\ [4,3] \end{matrix}$
$X[4,5]$	$\begin{matrix} W \\ [5,0] \end{matrix}$	$\begin{matrix} W \\ [5,1] \end{matrix}$	$\begin{matrix} W \\ [5,2] \end{matrix}$	$\begin{matrix} W \\ [5,3] \end{matrix}$
$X[4,6]$	$\begin{matrix} W \\ [6,0] \end{matrix}$	$\begin{matrix} W \\ [6,1] \end{matrix}$	$\begin{matrix} W \\ [6,2] \end{matrix}$	$\begin{matrix} W \\ [6,3] \end{matrix}$
$X[4,7]$	$\begin{matrix} W \\ [7,0] \end{matrix}$	$\begin{matrix} W \\ [7,1] \end{matrix}$	$\begin{matrix} W \\ [7,2] \end{matrix}$	$\begin{matrix} W \\ [7,3] \end{matrix}$

$Y[4,0] \ Y[4,1] \ Y[4,2] \ Y[4,3]$

Q2: 对于CIM我们希望减少weight的移动，这种方法移动的太多，如何解决？

A: 调整循环顺序，减少weight移动。实际上是WS (weight stationary) 数据流。



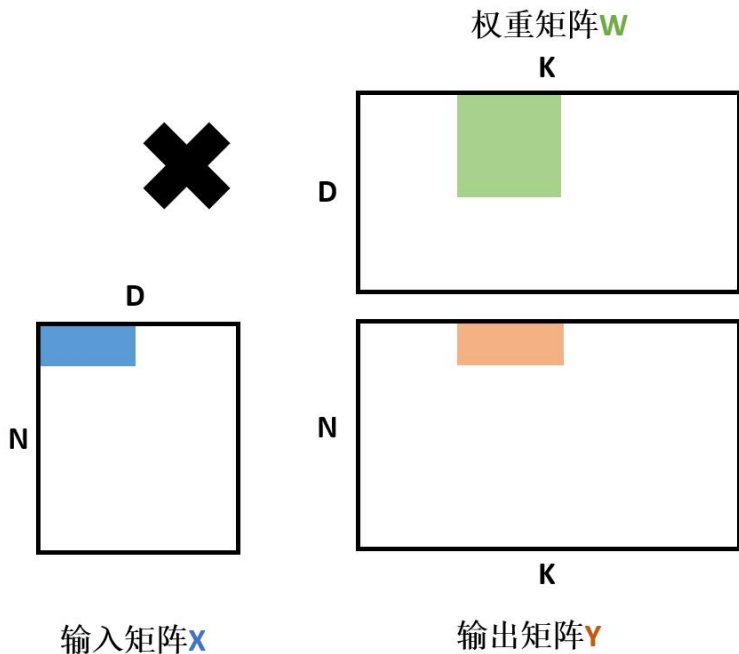
```
for (k1=0;k1<K1;k1++)
  for (d1=0;d1<D1;d1++)
    for (n=0;n<N;n++)
      if (d1==0)  $Y[n,k1*K0:(k1+1)*K0] = 0;$ 
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
           $Y[n,k1*K0+k0] += X[n,d1*D0+d0] * W[d1*D0+d0,k1*K0+k0];$ 
```

$X[N,4]$	$\begin{matrix} W \\ [4,0] \end{matrix}$	$\begin{matrix} W \\ [4,1] \end{matrix}$	$\begin{matrix} W \\ [4,2] \end{matrix}$	$\begin{matrix} W \\ [4,3] \end{matrix}$
$X[N,5]$	$\begin{matrix} W \\ [5,0] \end{matrix}$	$\begin{matrix} W \\ [5,1] \end{matrix}$	$\begin{matrix} W \\ [5,2] \end{matrix}$	$\begin{matrix} W \\ [5,3] \end{matrix}$
$X[N,6]$	$\begin{matrix} W \\ [6,0] \end{matrix}$	$\begin{matrix} W \\ [6,1] \end{matrix}$	$\begin{matrix} W \\ [6,2] \end{matrix}$	$\begin{matrix} W \\ [6,3] \end{matrix}$
$X[N,7]$	$\begin{matrix} W \\ [7,0] \end{matrix}$	$\begin{matrix} W \\ [7,1] \end{matrix}$	$\begin{matrix} W \\ [7,2] \end{matrix}$	$\begin{matrix} W \\ [7,3] \end{matrix}$

$Y[N,0] \ Y[N,1] \ Y[N,2] \ Y[N,3]$

Q2: 对于CIM我们希望减少weight的移动, 这种方法移动的太多, 如何解决?

A: 调整循环顺序, 减少weight移动。实际上这是WS (weight stationary) 数据流。



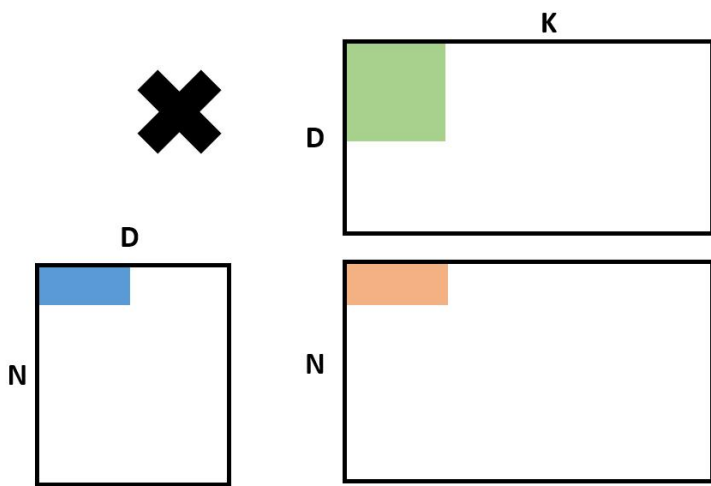
```
for (k1=0;k1<K1;k1++)
  for (d1=0;d1<D1;d1++)
    for (n=0;n<N;n++)
      if (d1==0) Y[n,k1*K0:(k1+1)*K0] = 0;
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
          Y[n,k1*K0+k0] += X[n,d1*D0+d0]*
            W[d1*D0+d0,k1*K0+k0];
```

$x[0,0]$	$\begin{matrix} W \\ [0,4] \end{matrix}$	$\begin{matrix} W \\ [0,5] \end{matrix}$	$\begin{matrix} W \\ [0,6] \end{matrix}$	$\begin{matrix} W \\ [0,7] \end{matrix}$
$x[0,1]$	$\begin{matrix} W \\ [1,4] \end{matrix}$	$\begin{matrix} W \\ [1,5] \end{matrix}$	$\begin{matrix} W \\ [1,6] \end{matrix}$	$\begin{matrix} W \\ [1,7] \end{matrix}$
$x[0,2]$	$\begin{matrix} W \\ [2,4] \end{matrix}$	$\begin{matrix} W \\ [2,5] \end{matrix}$	$\begin{matrix} W \\ [2,6] \end{matrix}$	$\begin{matrix} W \\ [2,7] \end{matrix}$
$x[0,3]$	$\begin{matrix} W \\ [3,4] \end{matrix}$	$\begin{matrix} W \\ [3,5] \end{matrix}$	$\begin{matrix} W \\ [3,6] \end{matrix}$	$\begin{matrix} W \\ [3,7] \end{matrix}$

$Y[0,4] \ Y[0,5] \ Y[0,6] \ Y[0,7]$

最小buffer开销下的数据传输比较, 最小buffer大小比较: 对CIM来说WS优于OS (更新开销大/算法 $N \gg D1$)

	Output data transmission (size*time)	Weight data transmission (size*time)	Input data transmission (size*time)	Minimum buffer size (O/W/I)
OS (output stationary)	$K0 * (N * K1) = N * K$	$(D0 * K0) * (D1 * K1 * N) = N * D * K$	$D0 * (D1 * K1 * N) = N * D * K1$	$K0 / D * K0 / D$
WS (weight stationary)	$K0 * (N * K1 * D1) = N * K * D1$	$(D0 * K0) * (D1 * K1) = D * K$	$D0 * (N * D1 * K1) = N * D * K1$	$K0 / D0 * K0 / D0$



OS

```

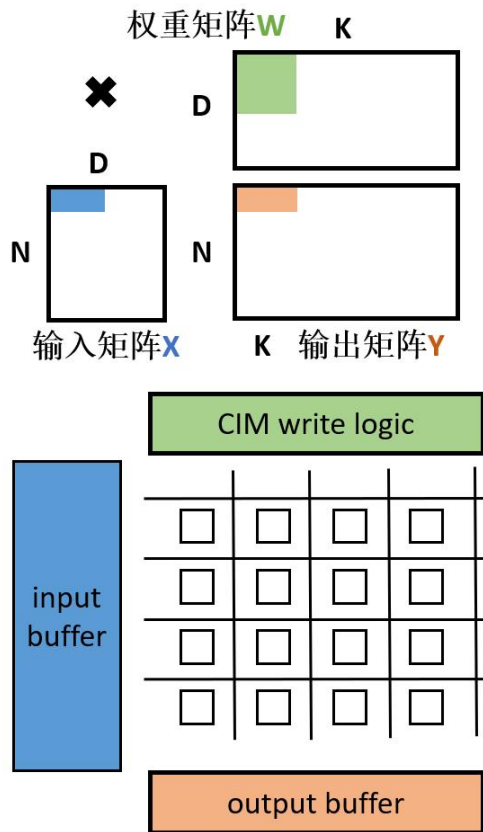
for (n=0;n<N;n++)
  for (k1=0;k1<K1;k1++)
    Y[n,k1*K0:(k1+1)*K0] = 0;
    for (d1=0;d1<D1;d1++)
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
          Y[n,k1*K0+k0] += X[n,d1*D0+d0] *
                           W[d1*D0+d0,k1*K0+k0];
  
```

WS

```

for (k1=0;k1<K1;k1++)
  for (d1=0;d1<D1;d1++)
    for (n=0;n<N;n++)
      if (d1==0) Y[n,k1*K0:(k1+1)*K0] = 0;
      spatial_for (k0=0;k0<K0;k0++) //column
        spatial_for (d0=0;d0<D0;d0++) //cell
          Y[n,k1*K0+k0] += X[n,d1*D0+d0] *
                           W[d1*D0+d0,k1*K0+k0];
  
```

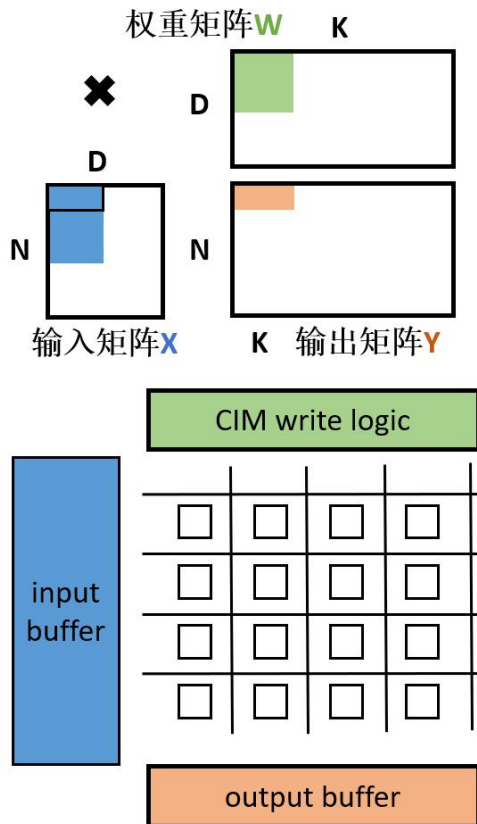
How to optimize for buffer size for WS architecture?



```

for (k1=0; k1<K1; k1++)
  for (d1=0; d1<D1; d1++)
    UpdateCIM(W[d1*D0:(d1+1)*D0, k1*K0:(k1+1)*K0]) //WB>=D0*K0
    for (n=0; n<N; n++)
      if (d1==0) Y[n, k1*K0:(k1+1)*K0] = 0;
      else mvin(Y[n, k1*K0:(k1+1)*K0]) //OB>=1*K0
      mvin(X[n, d1*D0:(d1+1)*D0]) //IB>=1*D0
      spatial_for (k0=0; k0<K0; k0++) //column
        spatial_for (d0=0; d0<D0; d0++) //cell
          Y[n, k1*K0+k0] += X[n, d1*D0+d0] *
                           W[d1*D0+d0, k1*K0+k0];
      mvout(Y[n, k1*K0:(k1+1)*K0])
  
```

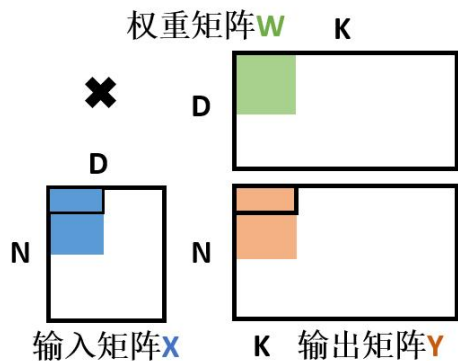
Q3: What if the input buffer size $> 1 \times D_0$? A: Stores $N_0 > 1$ rows of tiled input activations. 这里按列存储，考虑CIM size。



要求: $N_1 \times N_0 = N$ (loop bound)

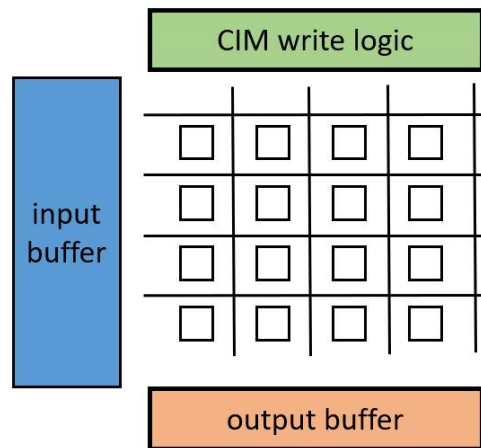
```
for (k1=0; k1<K1; k1++)
  for (d1=0; d1<D1; d1++)
    UpdateCIM(W[d1*D0:(d1+1)*D0, k1*K0:(k1+1)*K0]) //WB>=D0*K0
    for (n1=0; n1<N1; n1++)
      mvin(X[n1*N0:(n1+1)*N0, d1*D0:(d1+1)*d0]) //IB>=1*D0
      for (n0=0; n0<N0; n0++)
        if (d1==0) Y[n1*N0+n0, k1*K0:(k1+1)*K0] = 0;
        else mvin(Y[n1*N0+n0, k1*K0:(k1+1)*K0]) //OB>=1*K0
        spatial_for (k0=0; k0<K0; k0++) //column
          spatial_for (d0=0; d0<D0; d0++) //cell
            Y[n1*N0+n0, k1*K0+k0] +=
              X[n1*N0+n0, d1*D0+d0]*W[d1*D0+d0, k1*K0+k0];
      mvout(Y[n1*N0+n0, k1*K0:(k1+1)*K0])
```

Q4: What if the output buffer size $> 1 \times K_0$? A: Stores > 1 striped rows of output activations. 这里按列存储，考虑 CIM size。

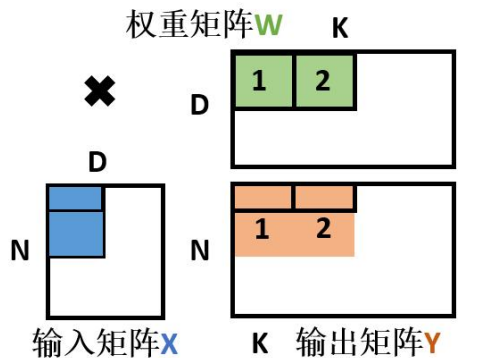


要求: $N_1 \times N_0 = N$ (loop bound)

```
for (k1=0; k1<K1; k1++)
  for (d1=0; d1<D1; d1++)
    UpdateCIM( $W[d1 \times D_0 : (d1+1) \times D_0, k1 \times K_0 : (k1+1) \times K_0]$ ) //WB>= $D_0 \times K_0$ 
    for (n1=0; n1<N1; n2++)
      mvin( $X[n1 \times N_0 : (n1+1) \times N_0, d1 \times D_0 : (d1+1) \times D_0]$ ) //IB>= $1 \times D_0$ 
      if (d1==0)  $Y[n1 \times N_0 : (n1+1) \times N_0, k1 \times K_0 : (k1+1) \times K_0] = 0;$ 
      else mvin( $Y[n1 \times N_0 : (n1+1) \times N_0, k1 \times K_0 : (k1+1) \times K_0]$ ) //OB>= $1 \times K_0$ 
      for (n0=0; n0<N0; n0++)
        spatial_for (k0=0; k0<K0; k0++) //column
          spatial_for (d0=0; d0<D0; d0++) //cell
             $Y[n1 \times N_0 + n0, k1 \times K_0 + k0] +=$ 
               $X[n1 \times N_0 + n0, d1 \times D_0 + d0] \times W[d1 \times D_0 + d0, k1 \times K_0 + k0];$ 
      mvout( $Y[n1 \times N_0 : (n1+1) \times N_0, k1 \times K_0 : (k1+1) \times K_0]$ )
```



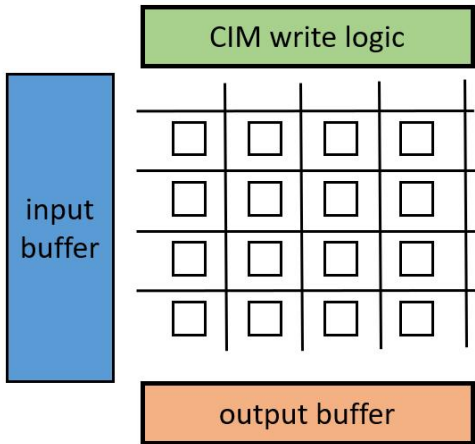
Q5: What if we have enough CIM array? K方向: 权重分布式存储, 输入广播, 输出全并行计算



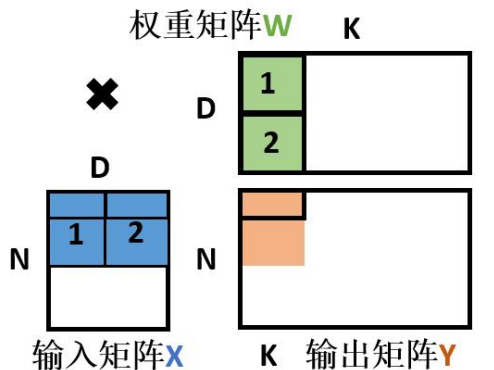
要求: $N_2 * N_1 = N$

```

spatial_for (k1=0;k1<K1;k1++)
for (d1=0;d1<D1;d1++)
  UpdateCIM(W[d1*D0:(d1+1)*D0,k1*K0:(k1+1)*K0]) //WB>=D0*K0
  for (n1=0;n1<N1;n2++)
    mvin(X[n1*N0:(n1+1)*N0,d1*D0:(d1+1)*d0]) //IB>=1*D0
    if (d1==0) Y[n1*N0:(n1+1)*N0,k1*K0:(k1+1)*K0] = 0;
    else mvin(Y[n1*N0:(n1+1)*N0,k1*K0:(k1+1)*K0]) //OB>=1*K0
    for (n0=0;n0<N0;n0++)
      spatial_for (k0=0;k0<K0;k0++) //column
      spatial_for (d0=0;d0<D0;d0++) //cell
        Y[n1*N0+n0,k1*K0+k0] +=
          X[n1*N0+n0,d1*D0+d0]*W[d1*D0+d0,k1*K0+k0];
    mvout(Y[n1*N0:(n1+1)*N0,k1*K0:(k1+1)*K0])
  
```

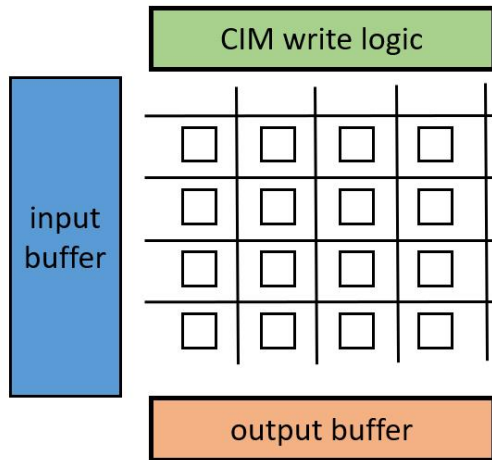


Q6: What if we have enough CIM array? type2: 权重分布式存储, 输入全并行, 输出累加。



要求: $N_2 * N_1 = N$

```
for (k1=0;k1<K1;k1++)
    spatial_for (d1=0;d1<D1;d1++)
        UpdateCIM( $W[d1*D_0:(d1+1)*D_0, k1*K_0:(k1+1)*K_0]$ ) //WB>=D0*K0
        for (n1=0;n1<N1;n2++)
            mvin( $X[n1*N_0:(n1+1)*N_0, d1*D_0:(d1+1)*d_0]$ ) //IB>=1*D0
             $Y[n1*N_0:(n1+1)*N_0, k1*K_0:(k1+1)*K_0]$ ; //OB>=1*K0
            for (n0=0;n0<N0;n0++)
                spatial_for (k0=0;k0<K0;k0++) //column
                    spatial_for (d0=0;d0<D0;d0++) //cell
                         $Y[n1*N_0+n_0, k1*K_0+k_0] +=$ 
                             $X[n1*N_0+n_0, d1*D_0+d_0] * W[d1*D_0+d_0, k1*K_0+k_0]$ ;
            mvout( $Y[n1*N_0:(n1+1)*N_0, k1*K_0:(k1+1)*K_0]$ )
            accumulate( $Y[n1*N_0:(n1+1)*N_0, k1*K_0:(k1+1)*K_0]$ )
```



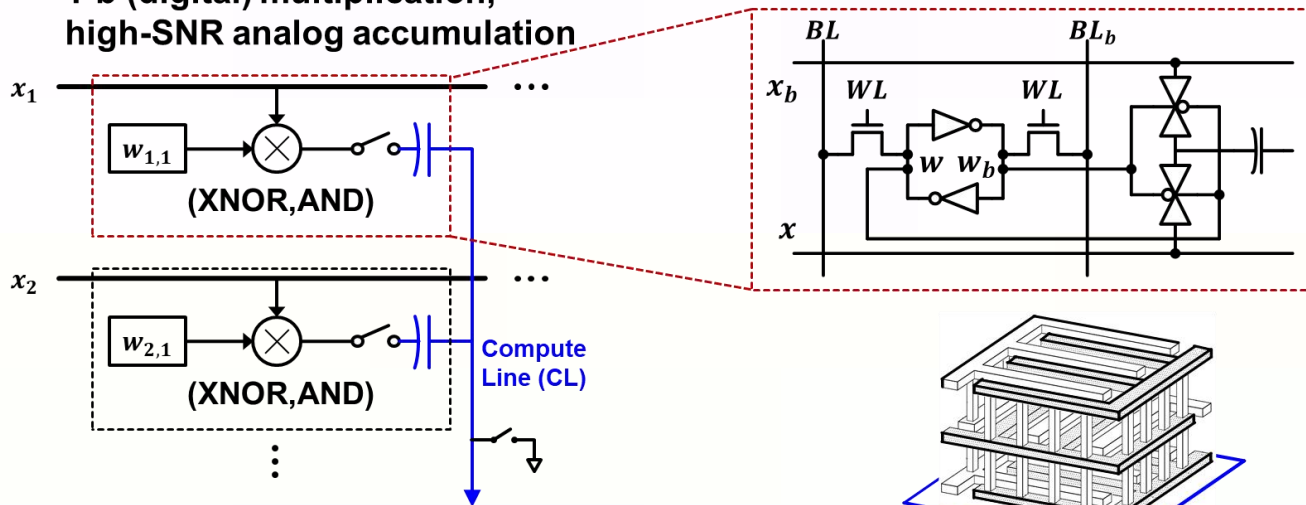
/Outlines

- **Introduction**
- **Memory devices and computational primitives**
 - **Charge-based memory devices**
 - **Resistance-based memory devices**
- **SRAM-based CIM: from device to architecture**
 - **3 SRAM CIM primitives**
 - **Toward efficient MVM mapping to CIM array**
 - **Sparsity in CIM**
- **Challenges, Trade-offs and Metrics of CIM**

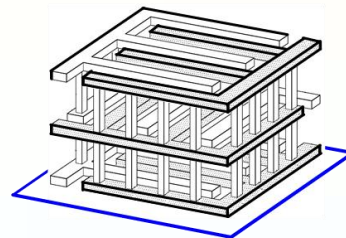
NN Algorithm on CIM: Sparsity

Native support of bit-level sparsity in charge-domain CIM

- 1-b (digital) multiplication, high-SNR analog accumulation



- **Capacitors precisely controlled in advanced CMOS technologies**
- **Total cell area overhead 1.8~2× vs. 6T cell**

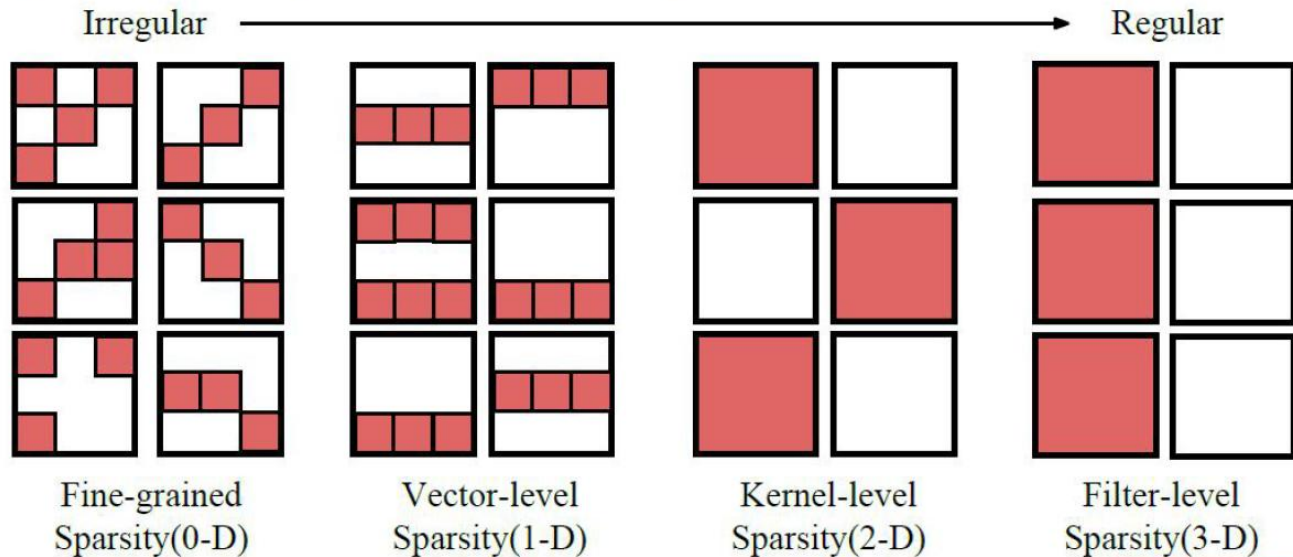


Metal cap. above bit cell

- 1.8× area of 6T cell
- ~10x smaller than equal digital circuit

/NN Algorithm on CIM: Sparsity

- **Fine-grain sparsity** — low utilization rate
- **Coarse-grain sparsity** — limited compression ratio

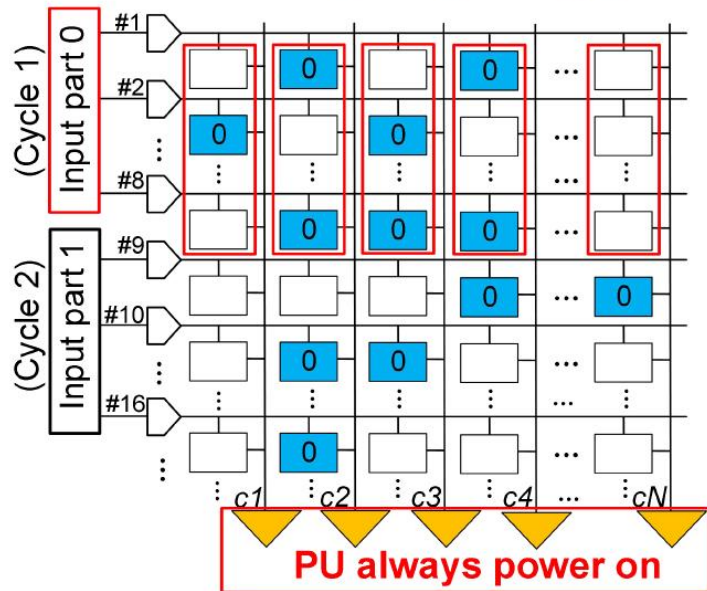


Compression ratio

Sparsity granularity

/NN Algorithm on CIM: Sparsity

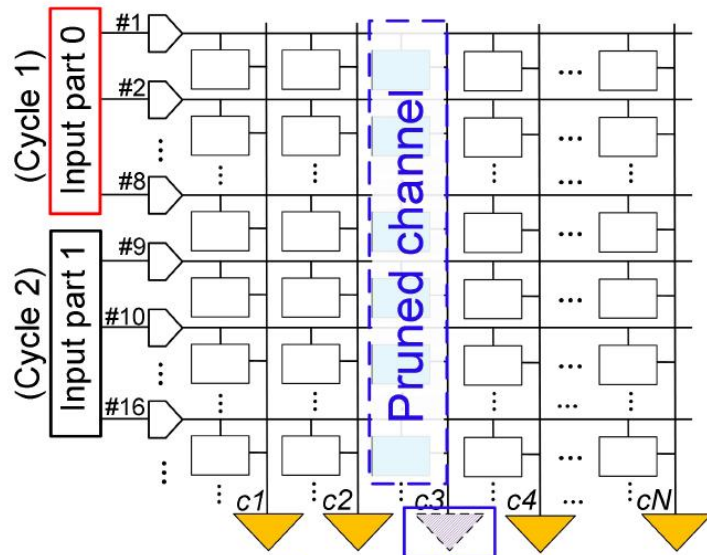
- Inefficient weight sparsity pattern in CIM architecture



Fine-grain sparsity

High compression ratio

Inefficient implementation



No need to implement

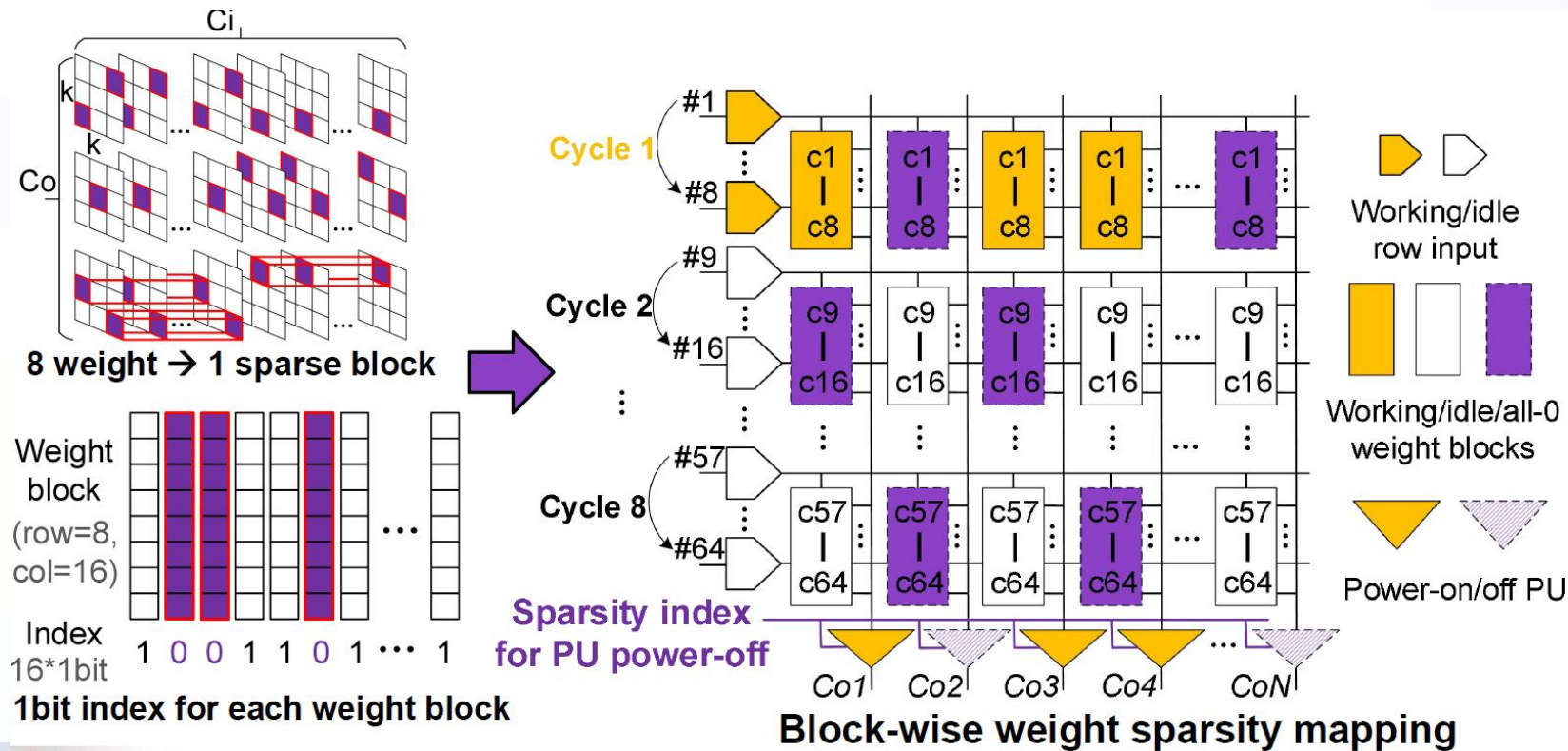
Structural sparsity (Filter-level)

Friendly implementation

Limited compression ratio

Source: ISSCC2020 14.3

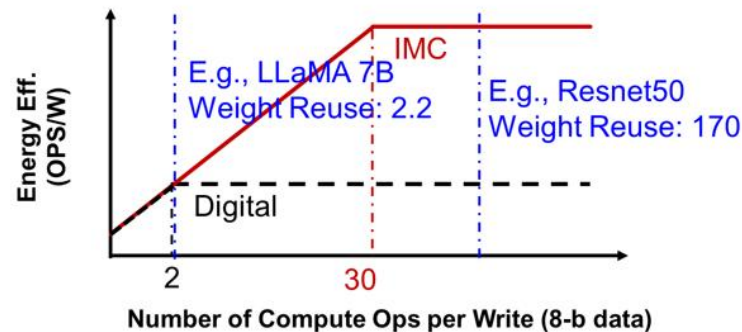
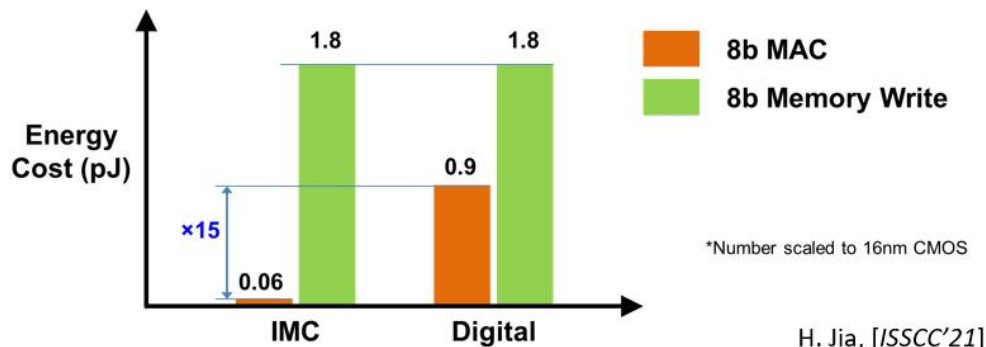
/NN Algorithm on CIM: Sparsity



/Outlines

- **Introduction**
- **Memory devices and computational primitives**
 - **Charge-based memory devices**
 - **Resistance-based memory devices**
- **SRAM-based CIM: from device to architecture**
 - **3 SRAM CIM primitives**
 - **Toward efficient MVM mapping to CIM array**
 - **Sparsity in CIM**
- **Challenges, Trade-offs and Metrics of CIM**

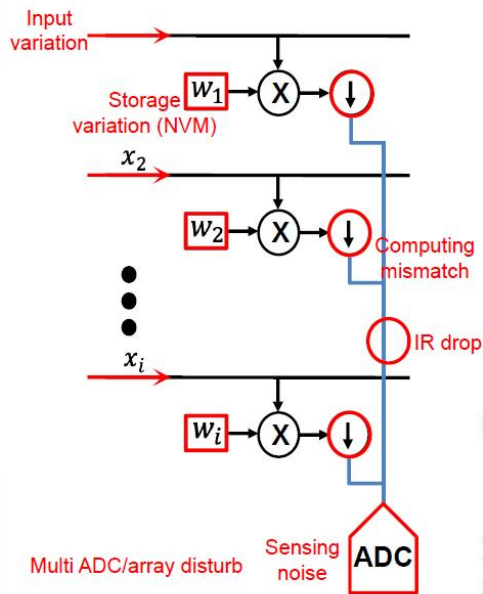
/Challenges of scalable IMC



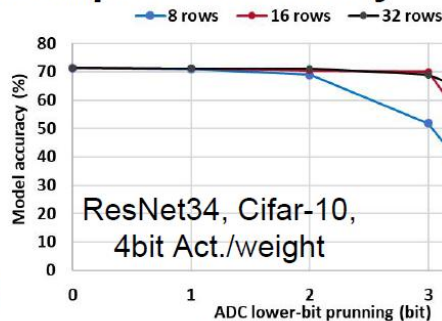
- IMC does not help memory write (NN weight loading)
- weight-writing costs must be adequately amortized (i.e. weight reuse)

/Accuracy Trade-Off in CIM

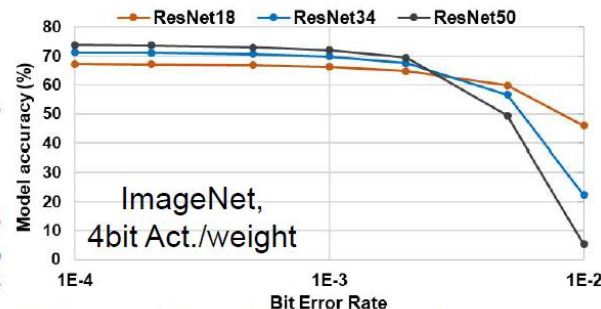
- Parallelism & ADC precision for power/accuracy trade-off
- Re-train is effective to improve accuracy



Variation in CIM circuits

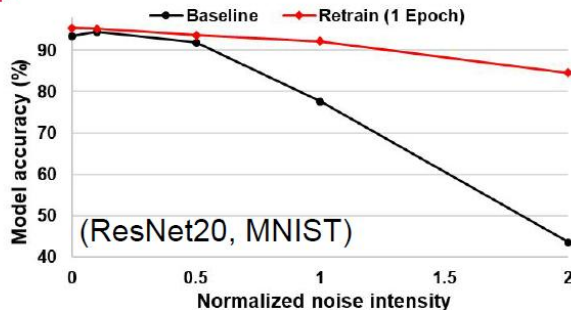


ResNet34, Cifar-10,
4bit Act./weight

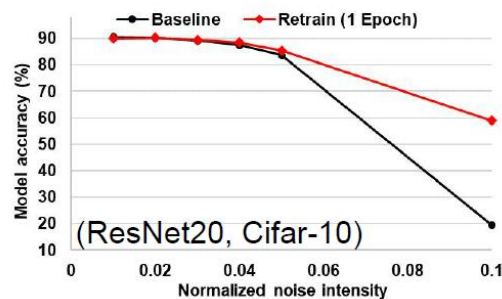


ImageNet,
4bit Act./weight

Accuracy vs. parallelism/ADC precision/Network depth



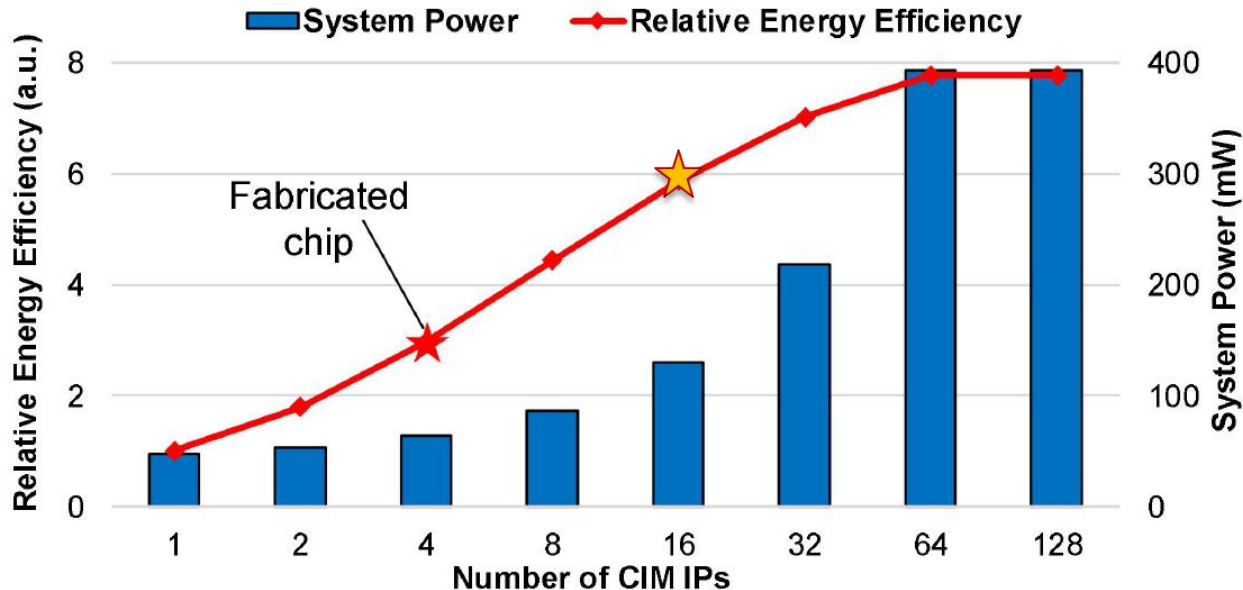
(ResNet20, MNIST)



(ResNet20, Cifar-10)

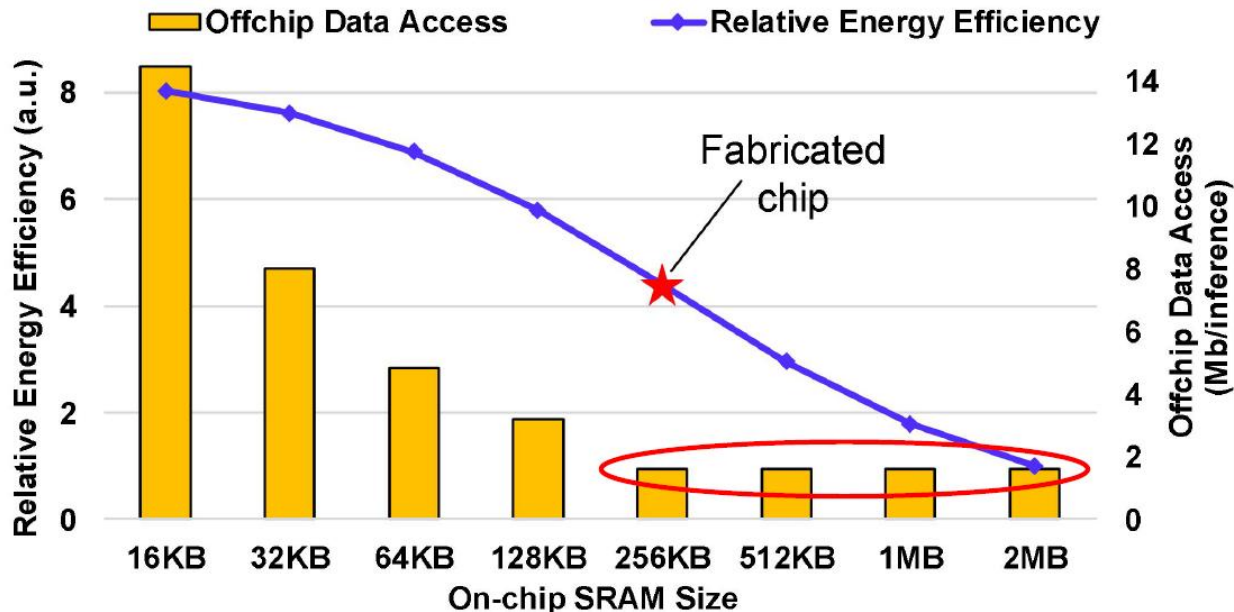
/Design Space Exploration in CIM

- Key of high energy efficiency: increase the power proportion of CIM IP
- Problems: system power/area limitation, off-chip data access, etc.
 - Increase CIM IP power (number) → increased system power/area



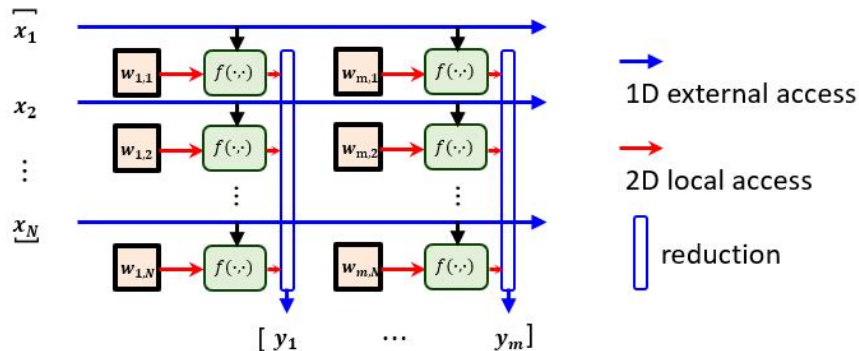
/Design Space Exploration in CIM

- Key of high energy efficiency: increase the power proportion of CIM IP
- Problems: system power/area limitation, off-chip data access, etc.
 - Reduce other on-chip power (SRAMs) → increased off-chip data access



/ The fundamental gain from IMC

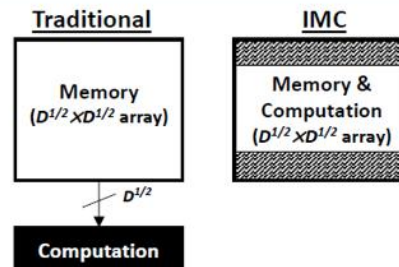
Dataflow



- 2D highly parallel local data access
- 2D highly parallel local compute
 - Multiplication in MVM; different in other norms
- Reduction operation
 - Convert 2D external data movement to 1D
 - Summation, max, ...

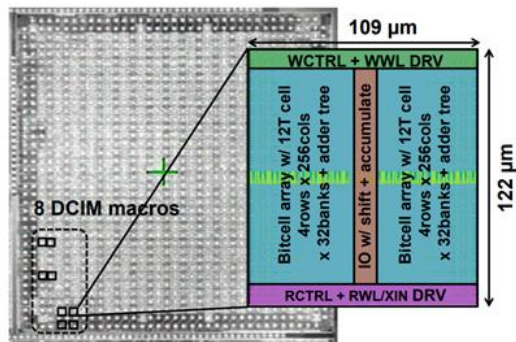
Physical design

Metric	Traditional	In-memory
Bandwidth	$1/D^{1/2}$	1
Latency	D	1
Energy	$D^{3/2}$	$\sim D$



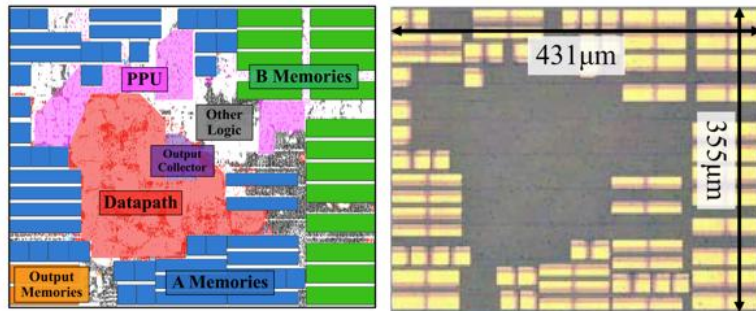
- Tightly coupled memory bits and compute
- Treat transistors as more than switches in A-IMC

Understanding the benefits of fully digital IMC



Technology	5 nm HKMG FinFET
Bitcell	Low voltage 12T (0.075 μm^2)
Macro size	109 μm x 122 μm
Memory config.	64 kb 256 rows x 256 IOs w/ MUX1
MAC size	1024 MAC / CLK cycle Input: 64 x 4b Weight: 4b w/ 4rows Output: 64 x 14b

Input w/ signed or unsigned
Weight w/ signed or unsigned



[B. Keller, Nvidia, JSSC'23]

Confirmed ~1/3 area of automatic P&R result.

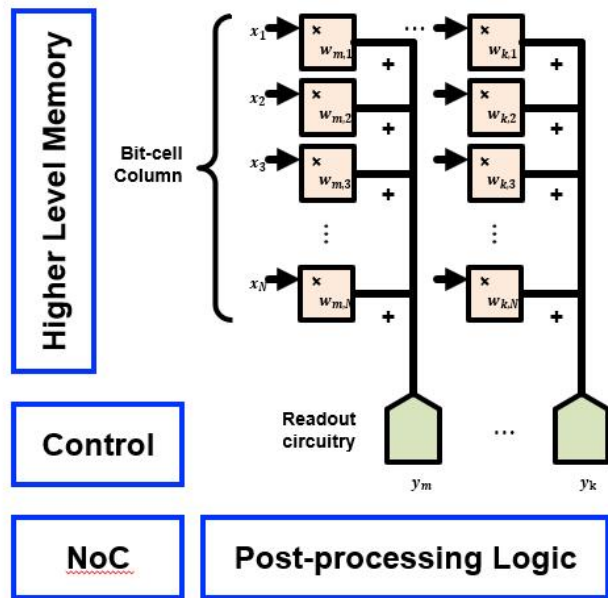
[H. Fujiwara, TSMC, ISSCC'22]

- D-IMC gains from customized layout and compact SRAM cell (compared to DFF).
- 3x area reduction over APR result

- 5nm D-IMC macro [H. Fujiwara, TSMC, ISSCC'22]
 - ~150 4b-TOPS/W @0.5V (50% sparsity), 221 4b-TOPS/ mm^2 @0.9V
- 5nm Digital PE [K. Ben, Nvidia, JSSC'23]
 - ~90 4b-TOPS/W @0.46V (50% sparsity), 24 4b-TOPS/ mm^2 @1.1V

Transformer support and Data buffer included
>100 4b-TOPS/ mm^2 when consider datapath only

/Energy metrics of IMC



**So many energy metrics,
which to follow?**

- **Core energy efficiency**
 - ☐ Input broadcast; Control broadcast
 - ☐ Local memory access
 - ☐ Local Multiplication
 - ☐ Reduction and readout
- **'System' energy efficiency**
 - ☐ Higher level memory
 - ☐ PPU, Communication, Control
- **Peak and average energy efficiency**
 - ☐ Depend on application dataflow
 - ☐ Batch or single run?
- **Effective energy efficiency**
 - ☐ Depend on algorithm and data
 - ☐ Block floating point; Approx.; ...

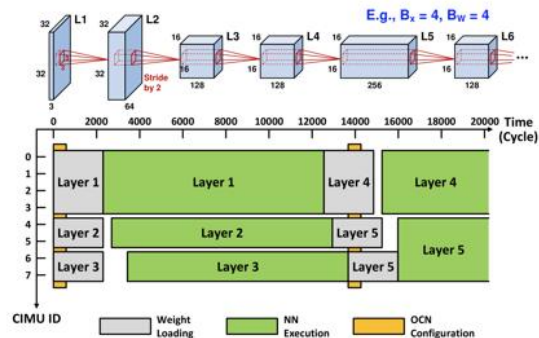
/ Performance metrics of IMC

Throughput vs Latency

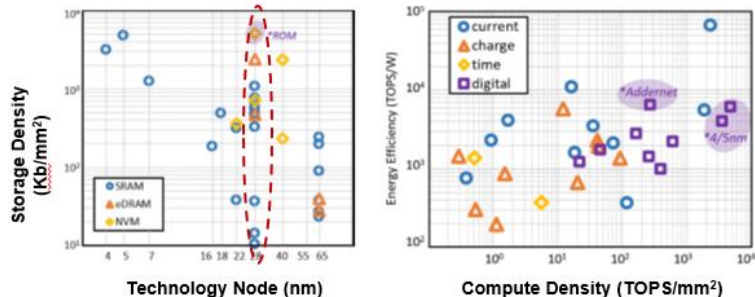
- Latency >> Throughput
 - ❑ Throughput is additive, latency is not
 - ❑ Hiding of memory write
 - ❑ Hiding of communication and config
 - ❑ Pay by system compute density...

Compute Density vs Storage Density

- An important tradeoff
 - ❑ Memory or accelerator?
 - ❑ Storage should dominate the area, otherwise...



[H. Jia, JSSC'22]



[W. Sun, OJ-SSCS'23]

/ Application-level metrics of IMC

Utilization vs Energy-efficiency

Given: 800mm², 5 TOPS/mm², 20 TOPS/W

IMC chip: 4000 TOPS, 0.2kW (peak)

Total cost of ownership (TCO):

Chip + board + infra. (Flat)

+

Electricity (Both chip and cooling...)

+

...

□ 100% Utilization:

Electricity = 5 year × 365 day × 24 h × 0.2 kW × 0.6 CNY/(kW·h) × 3 ≈ 16000 CNY

Flat ≈ 20000 CNY for example

Cost = 0.057 CNY/EOP

□ 20 TOPS/W → 100 TOPS/W:

Cost = 0.037 CNY/EOP

Utilization is critical

□ 100% Utilization → 50% Utilization:

Cost = 0.089 (gating) ~ 0.114 CNY/EOP

Evaluation of Service Quality

Dataset	CIFAR-10	ImageNet
Neural-network style	VGG	ResNet-50
Bit precision	4b/4b weight/activation	4b/4b weight/activation ¹
Accuracy of chip (vs sim.)	91.51% (vs. 91.68%)	73.33% (vs. 73.89%)
Frame per second ²	7815 images/sec.	581 images/sec.
Avg. throughput ²	9.8 TOPS	3.4 TOPS
Accelerator energy eff. ²	51.5K image/sec/W	3.0K image/sec/W
IMC bit-cell utilization ³	83%	29%

[H. Jia, /ISSCC'21]

Input precision (bit)	8	BF16
Weight precision (bit)	8	BF16
Energy efficiency (TOPS/W) (TFLOPS/W) (¹ Peak performance)	125.6-163.3	72.3-91.2
Area efficiency (TOPS/mm ²) (TFLOPS/mm ²)	11.07 (0.8V)	2.63 (0.8V)
² Inference Accuracy Loss (CIFAR-100)	-0.19%	-0.01%
³ Inference Accuracy Loss (ImageNet)	-0.58%	-0.02%

¹Performance in 90% input sparsity, 10% input toggle rate (ResNet-20)

²Using ResNet-20 model and the software baseline (FP32) was 70.22%

³Using ResNet-18 model and the software baseline (FP32) was 68.31%

[W.-S. Khwa, /ISSCC'24]

Data Precision	Posit(8,1)	Posit(16,2)
Cim Micro Energy Efficiency ²⁾	16.34-83.23 TOPS/W	7.47-38.37 TOPS/W
System Energy Efficiency ²⁾	10.90-55.60 TOPS/W	5.35-27.61 TOPS/W
Different AI Models	ResNet18 ²⁾ @Imagenet1k	69.64% (Top1 Acc†)
	GPT-2 ⁴⁾ @Wikitext-2	21.31 (Perplexity↓)
	VIT-B ³⁾ @Imagenet1k	80.17% (Top1 Acc†)
		69.71% (Top1 Acc†)

[Y. Wang, /ISSCC'24]

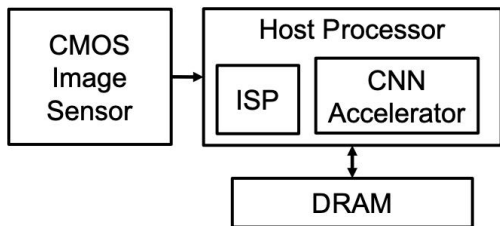
Require efficient and unified SW/HW environment: Compiling and benchmarking

/Processing-in-Sensor

Our Approach

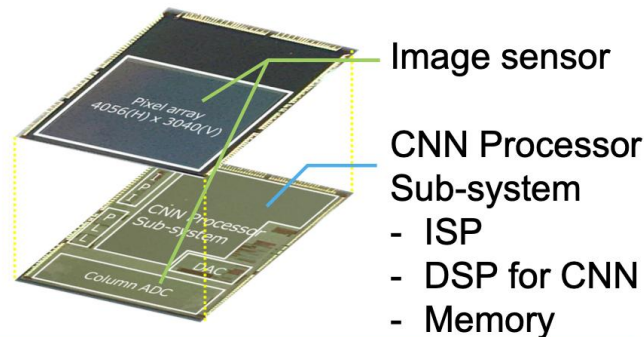
AI processing capability on edge devices solves issues of cloud system.

Conventional Approach



Larger, higher power, more costly

This Work's Approach

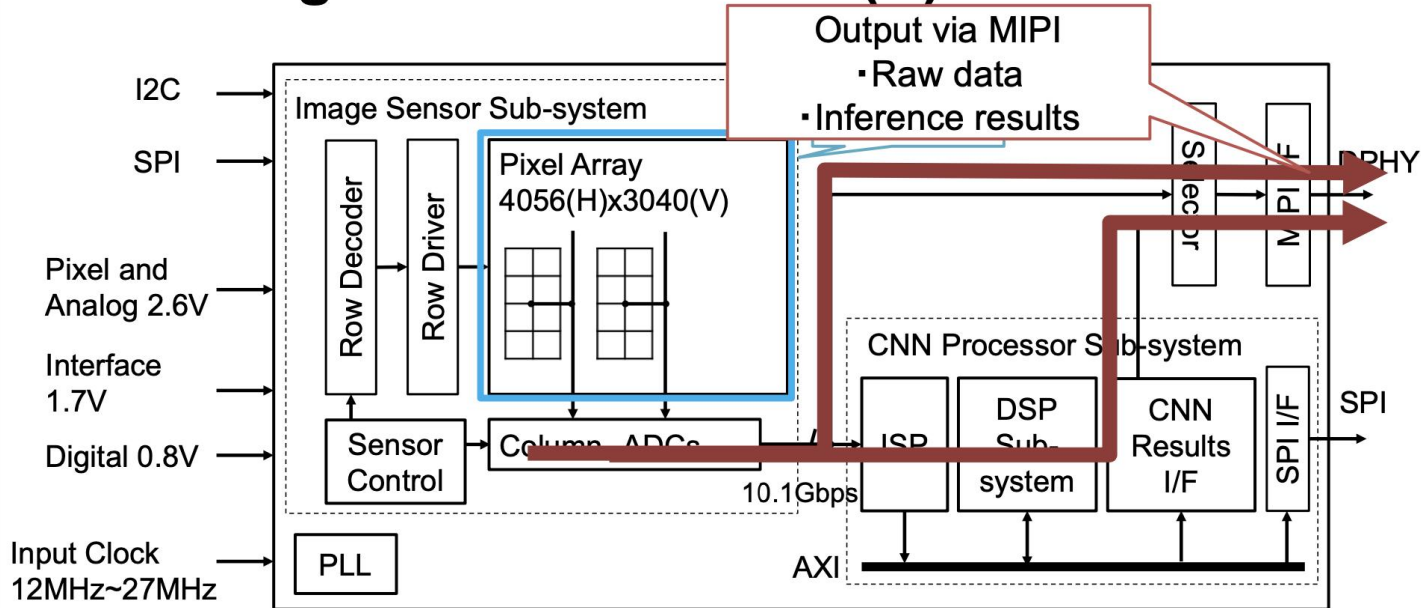


All functions are implemented on a chip

Intelligent Vision Sensor contributes to realizing full AI processing capability with “**Small form factor**”, “**Low system power and cost**”, “**Improved privacy**”

/Processing-in-Sensor

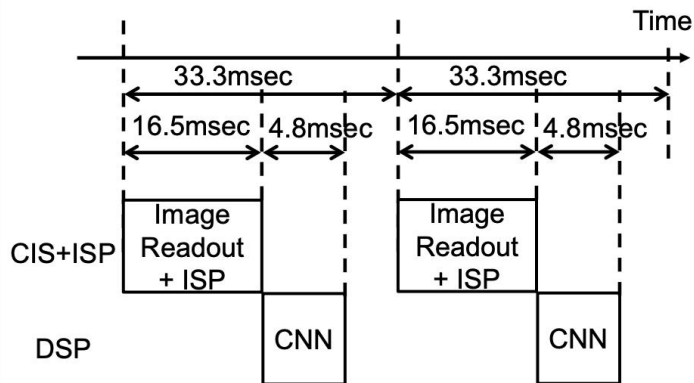
Block Diagram and Data Path(A)



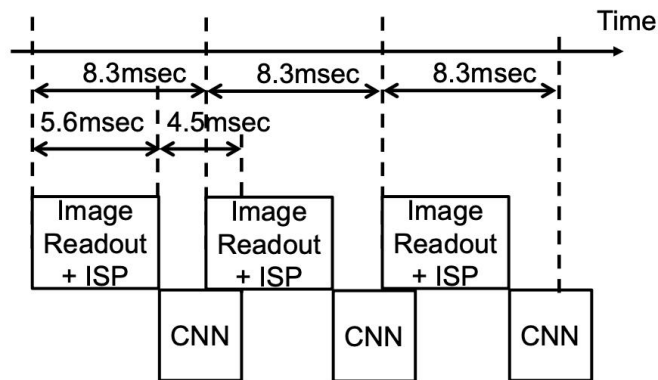
/Processing-in-Sensor

Timing Chart

Mode A: 12.3M[4056(H)x3040(V)] 30fps



Mode B: 3.1M [2028(H)x1520(V)] 120fps



Condition: CNN=Mobilenet_V1_1.0_224_quant
DSP Clock=262.5MHz

/Review

- Introduction: **CIM to break the memory wall**
- Memory devices and computational primitives
 - Charge-based memory devices: **SRAM, DRAM, Flash**
 - Resistance-based memory devices: **ReRAM, PCM, STT-MRAM**
- SRAM-based CIM: from device to architecture
 - 3 SRAM CIM primitives: **current, charge, digital**
 - Toward efficient MVM mapping to CIM array: **OS, matrix mapping**
 - Sparsity in CIM: **structural sparsity for CIM**
- Challenges, Trade-offs and Metrics of CIM: **memory write, accuracy, etc.**